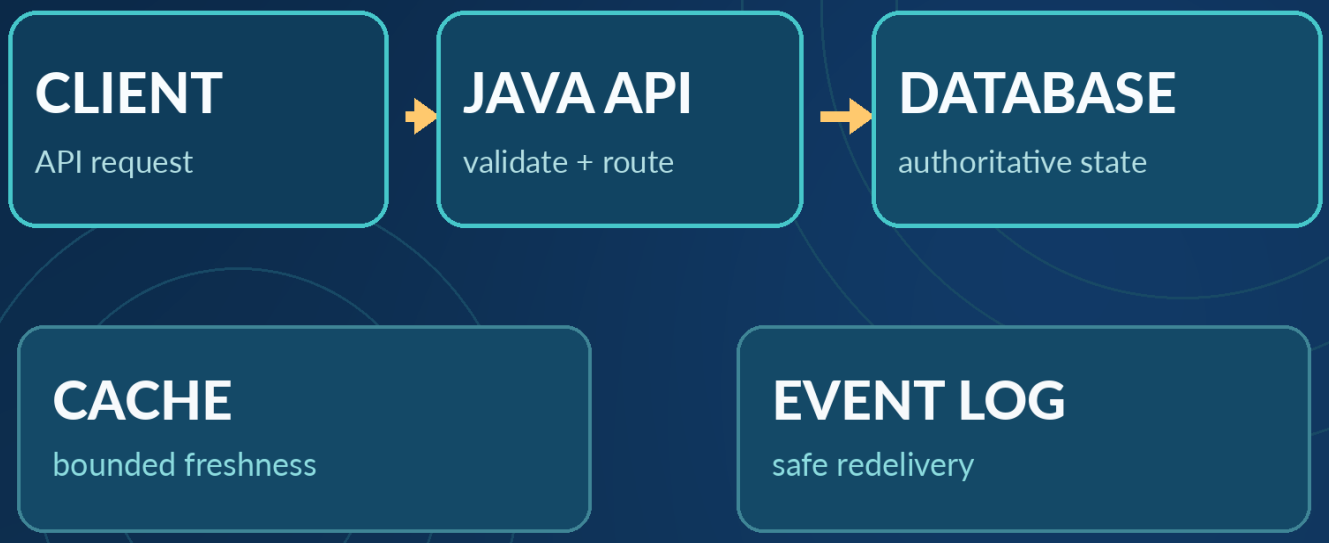
PREMIUM INTERVIEW NOTEBOOK

SYSTEM DESIGN

ZERO TO HERO

Java + Architecture Interview Guide

Clear answers. Visual systems. Real failure paths.



REQUIREMENTS > GUARANTEES > FAILURES

120 ORIGINAL QUESTIONS | 120 FOLLOW-UPS

8 WORKED ARCHITECTURES | 12 JAVA LABS

30-DAY STUDY ROUTE + INTERVIEW REVISION

JAVA 17 LABS • CLEAN UNRULED EDITION

How to use this notebook

For each question, learn the shortest correct answer first. Next, trace its diagram, speak through the example, explain the failure risk, and answer the follow-up aloud. This guide is organized around **what must remain true when components fail.**

120	120	8	12
original questions	follow-ups answered	worked architectures	verified Java labs

What each study page contains

Short answer	Interview-ready explanation without unnecessary jargon
Visual flow	Follow responsibilities and critical decisions
Example	Original SQL, Java, API example or scenario
Why it works	The actual guarantee and its boundary
Common pitfall	What could go wrong or be misunderstood
Follow-up	Original interviewer prompt plus a short answer

Important scope: Architecture and framework examples are explanatory sketches. The 12 companion Java labs are standalone Java 17 programs. A passing local test does not establish distributed-system durability or production performance.

Suggested study loop

Explain the requirement in 30 seconds. Draw the critical path in 60 seconds. Walk through one failure. State the tradeoff and a test that would verify your promise. Then solve or modify the closest Java lab.

Contents and study path

01 How to approach a system design interview

02 Requirements, estimates and capacity

03 Networking and API contracts

04 Scaling and load distribution

05 Choosing storage and data models

06 Transactions and concurrency control

07 Replication, partitioning and hot keys

08 Consistency, CAP and consensus

09 Caching and content delivery

10 Queues, logs and streaming

11 Outbox, idempotency and distributed workflows

12 Timeouts, retries and resilience

13 Java concurrency and resource budgets

14 Service boundaries and architecture patterns

15 Search, analytics and derived data

16 Realtime connections and fan-out

17 Security and tenant isolation

18 Observability and performance diagnosis

19 Deployment, shutdown and disaster recovery

20 Multi-region and global systems

21 Rate limiting, fairness and admission

22 Identifiers and useful algorithms

23 Business invariants: money, stock and reservations

24 Senior deep dives and production scenarios

Architecture workbook: URL shortener; distributed rate limiter; checkout; ticket booking; durable chat; social feed; large-file upload; notification platform.

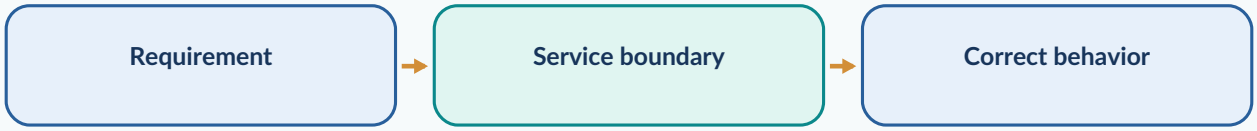
Code workbook: Token bucket, LRU, TTL, rendezvous hashing, executor bulkhead, single-flight, idempotency, inventory CAS, cursor pagination, fencing, reservations and outbox redelivery.

What is system design, and how is it different from coding?

SHORT INTERVIEW ANSWER

System design decides how components, data and operations work together to meet functional and quality requirements. Coding implements behavior inside those boundaries.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Java method: reserve(productId, quantity)
System question: who owns stock, what happens on retry, and how do we prevent two orders selling the last unit?

WHY THIS WORKS

A correct method can still belong to an unreliable system. The design must explain storage, concurrent requests, timeouts, recovery and ownership. A diagram is useful only when you can explain a request and a failure through it.

COMMON PITFALL Starting with Kafka or Kubernetes before describing the requirement usually hides the main problem.

INTERVIEWER FOLLOW-UP

Which invariant must survive a service crash?

Answer: Stock cannot go below zero and one seat cannot belong to two buyers. Enforce these rules in durable database transactions.

What should you clarify before drawing the architecture?

SHORT INTERVIEW ANSWER

Clarify the main operations, scale, latency, durability, consistency, security and scope. Separate requirements from assumptions and confirm which user journey matters most.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Ticket booking assumptions:
Search may be slightly stale.
A confirmed seat must have one owner.
Payment timeout must remain recoverable.

WHY THIS WORKS

These requirements allow a cached search path but require an authoritative booking transaction. Ask about peak traffic, geographic scope and retention. When the interviewer does not provide numbers, use explicit hypothetical values and revise the design if those values change.

COMMON PITFALL Do not treat every endpoint as needing identical consistency or availability.

INTERVIEWER FOLLOW-UP

Is an unavailable checkout preferable to duplicate sales?

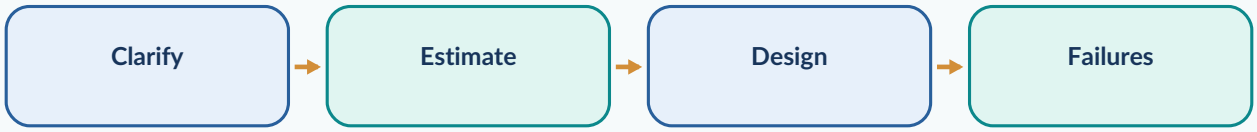
Answer: Usually yes: reject or delay checkout instead of making duplicate sales. Show the user a clear retry message.

How should you use a 45-minute design interview?

SHORT INTERVIEW ANSWER

Spend a few minutes defining the problem, estimate the critical scale, draw a minimal complete design, then deepen the highest-risk decisions and explain failures.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

0-5 min: scope and invariants
5-10: workload, APIs and data model
10-20: request path and architecture
20-38: concurrency, scaling and failure deep dive
38-45: tradeoffs, monitoring and evolution

WHY THIS WORKS

Treat this schedule as a practice aid. If the interviewer asks about partitioning early, adapt. A strong answer connects decisions: the workload motivates a partition key, that key creates a hot-key risk, and the mitigation changes cost or ordering.

COMMON PITFALL Listing components without walking through a request does not demonstrate correctness.

INTERVIEWER FOLLOW-UP

Where is the first bottleneck at ten times the traffic?

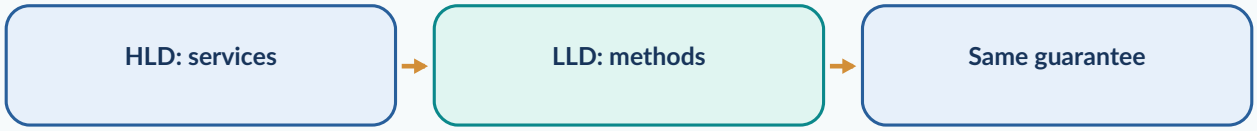
Answer: Measure database queries, hot keys, external calls and pool usage under a 10x load test; optimize the first saturated part.

What is the difference between high-level and low-level design?

SHORT INTERVIEW ANSWER

High-level design describes service boundaries, storage, communication and deployment. Low-level design describes classes, interfaces, algorithms and state transitions within a component.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
HLD: Booking service owns holds in a relational database.
LLD: enum HoldState { ACTIVE, CONFIRMED, EXPIRED }
Java command: confirm(holdId, paymentReference)
```

WHY THIS WORKS

The levels should agree. If the HLD promises idempotent confirmation, the LLD needs a stable command key and duplicate handling. Avoid spending the whole interview on class diagrams when the central challenge is distributed recovery.

COMMON PITFALL A Java interface does not define a network reliability contract by itself.

INTERVIEWER FOLLOW-UP

What happens if confirm arrives twice?

Answer: Use the same request ID. Read the stored confirmation result; never charge or reserve the same seat again.

What makes an architecture answer convincing?

SHORT INTERVIEW ANSWER

State the invariant, explain the normal path, expose the failure path, quantify assumptions and justify the tradeoff. Show how you will observe and evolve the design.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Claim: no overselling.
Mechanism: conditional stock update in one DB transaction.
Evidence: concurrent test and affected-row count.
Boundary: external payment is a separate workflow.

WHY THIS WORKS

This answer is stronger than saying that a database is ACID. It identifies the operation that enforces the guarantee and the point at which that guarantee stops. Include one rejected alternative and why it conflicts with the workload.

COMMON PITFALL Avoid promising absolute availability, zero latency or universal exactly-once processing.

INTERVIEWER FOLLOW-UP

How would you prove the invariant under concurrent requests?

Answer: Run many simultaneous purchase requests with one unit in stock; assert exactly one reservation and stock never falls below zero.

How do you convert daily requests into peak requests per second?

SHORT INTERVIEW ANSWER

Divide daily requests by 86,400 to get the average, then apply an explicit peak factor. Estimate reads and writes separately because they stress different resources.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Assume 20,000,000 reads/day and 2,000,000 writes/day.
Average reads = 231.5/s; writes = 23.1/s.
At 8x peak: reads = 1,852/s; writes = 185/s.
```

WHY THIS WORKS

These numbers suggest a moderate write load and a read-heavy workload, but they do not determine machine count. Benchmark the actual request path, payload, query and failure behavior. Include bursts such as a sale opening rather than assuming a smooth average.

COMMON PITFALL A peak multiplier is a stated assumption, not a universal industry constant.

INTERVIEWER FOLLOW-UP

What if a single key receives half the reads?

Answer: Cache or copy hot read-only data and use request coalescing. Do not split authoritative stock without coordination.

How do you estimate storage and bandwidth?

SHORT INTERVIEW ANSWER

Multiply retained record count by average encoded size, then add indexes, replicas and operational overhead. For bandwidth, multiply requests per second by transferred bytes.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

1,000,000 events/day x 500 bytes x 30 days = 15 GB.
Three copies: approximately 45 GB of raw event data.
2,000 responses/s x 10 KB = approximately 20 MB/s.

WHY THIS WORKS

This uses decimal units and excludes indexes, protocol overhead, compression and backups. Java heap object size can be much larger than serialized size. A design should distinguish working-set memory, authoritative storage, log retention and backup capacity.

COMMON PITFALL Do not equate three replicas with three independent backups.

INTERVIEWER FOLLOW-UP

How much extra space is needed during rebalancing?

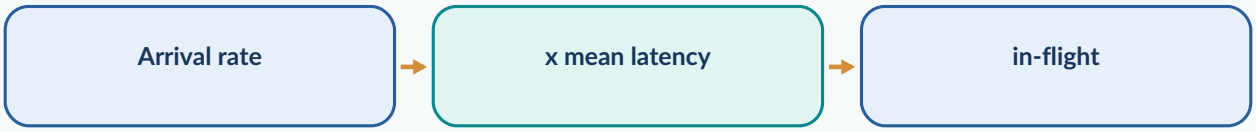
Answer: Allow spare capacity for temporary duplicate copies plus indexes, compaction and safety margin. Estimate from the rebalancing plan.

What does Little's Law tell you about concurrency?

SHORT INTERVIEW ANSWER

For a stable system, average in-flight work equals arrival rate multiplied by average time in the system. It helps estimate concurrency and recognize queue growth.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Arrival rate = 1,000 requests/s.
Mean response time = 0.2 seconds.
Average in-flight requests = 200.

WHY THIS WORKS

The time includes waiting when your boundary includes the queue. This relationship does not directly size a thread pool: requests may wait asynchronously or use several downstream resources. When arrivals exceed completion capacity for a sustained period, the stable-system assumption fails and queues grow.

COMMON PITFALL Using p99 latency in place of the mean does not produce average concurrency.

INTERVIEWER FOLLOW-UP

How does a slower database change in-flight work?

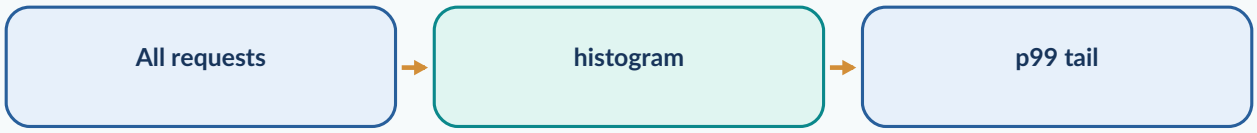
Answer: For equal arrival rate, a slower database raises average in-flight work (L = arrival rate x response time) and may grow queues.

Why do p95 and p99 matter more than average latency alone?

SHORT INTERVIEW ANSWER

Percentiles describe how slow requests affect a portion of users. An acceptable mean can hide a long tail caused by pauses, queueing, lock contention or slow dependencies.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

For 10,000 sorted request durations:
p99 is near the 9,900th duration.
Mean 40 ms with p99 2,000 ms can still hurt checkout.

WHY THIS WORKS

Measure the complete user operation and important internal stages. Fan-out often increases the chance that at least one dependency is slow. Keep histogram boundaries suitable for the SLO, and aggregate distributions rather than averaging percentile values across servers.

COMMON PITFALL An average of instance p99 values is not the fleet p99.

INTERVIEWER FOLLOW-UP

Does the latency include queue and connection acquisition time?

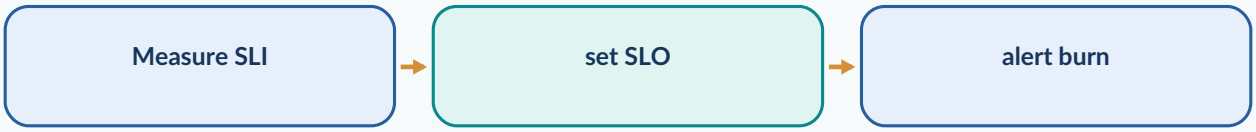
Answer: Yes. Measure from arrival to final response, including waiting for queues, a connection and downstream services.

How do SLI, SLO and SLA differ?

SHORT INTERVIEW ANSWER

An SLI is a measured service behavior, an SLO is its target, and an SLA is an agreement with consequences. Define success from the user's perspective.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

SLI: successful valid checkout requests / valid requests.
SLO example: 99.9% over a rolling 30-day window.
At 1,000,000 eligible requests: budget = 1,000 failures.

WHY THIS WORKS

Document which errors count and how the denominator is measured. A request-based budget is different from a time-based downtime allowance. A fast response containing an unusable result may still be a failure. Use burn rate to detect rapid budget consumption.

COMMON PITFALL Do not exclude overload failures simply to improve the reported success rate.

INTERVIEWER FOLLOW-UP

What signal detects a bad deploy before the monthly SLO fails?

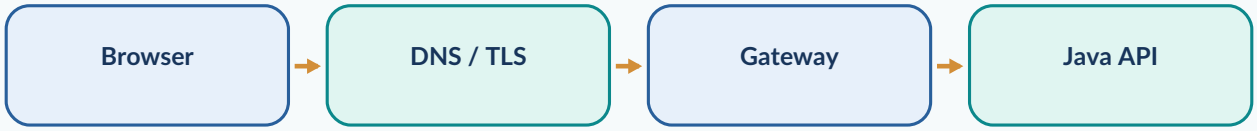
Answer: Watch short-window error rate, p99 latency and error-budget burn during a canary; roll back early if they spike.

What happens between entering a URL and reaching a Java service?

SHORT INTERVIEW ANSWER

The client resolves a name, establishes or reuses a connection, negotiates transport security when needed, and sends an HTTP request through routing layers to the application.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Browser -> DNS -> edge/CDN -> load balancer
-> Spring controller -> service -> repository
```

WHY THIS WORKS

Connection reuse, DNS caching and HTTP version change the exact sequence. TLS protects transport but does not replace authorization. Trace the time spent before the controller as well as inside it; a controller log alone cannot reveal a DNS or handshake failure.

COMMON PITFALL Assuming every request creates a fresh TCP connection gives misleading latency estimates.

INTERVIEWER FOLLOW-UP

Where can a timeout occur before Java receives the request?

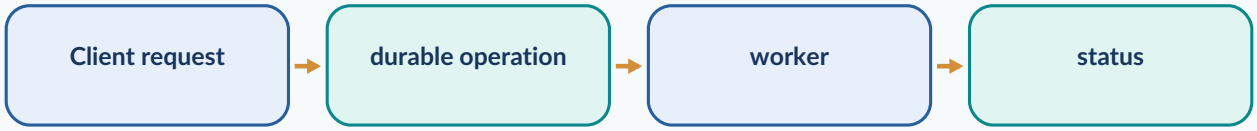
Answer: DNS lookup, connection setup, TLS handshake, proxy routing and the load balancer can all time out before the controller.

When should you choose REST, RPC or asynchronous messaging?

SHORT INTERVIEW ANSWER

Choose according to interaction semantics. Resource-oriented HTTP works well for public APIs; typed RPC helps internal request-response calls; messages decouple completion when the caller can accept a delayed outcome.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
POST /orders -> 202 + operation URL for long work.
GET /operations/op-42 -> PENDING or COMPLETED.
OrderCreated event -> independent notification consumer.
```

WHY THIS WORKS

The operation record must be durable before acknowledging accepted work. RPC still needs timeouts, compatibility and authorization. Messaging shifts complexity to delivery, ordering, retention and status tracking; it does not remove it.

COMMON PITFALL Returning 202 before work is durably accepted can lose an acknowledged operation.

INTERVIEWER FOLLOW-UP

How can the client discover the final outcome?

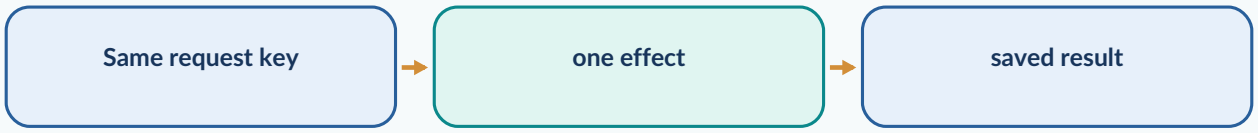
Answer: Return an operation ID; support GET /operations/{id}. Optionally send a webhook or event when finished.

What does HTTP idempotency mean?

SHORT INTERVIEW ANSWER

Repeating an idempotent operation has the same intended effect as performing it once. Response codes or audit entries may differ. POST can gain application-level idempotency through an explicit contract.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
PUT /profiles/42 with the same representation:
intended stored representation stays the same.
POST /payments with Idempotency-Key: pay-7:
the service must define duplicate handling.
```

WHY THIS WORKS

HTTP method semantics describe intent, not proof of an implementation. For a payment API, bind a key to the caller and a validated request fingerprint, retain the result, and reject key reuse with different intent. Scope and retention are part of the contract.

COMMON PITFALL Generating a new idempotency key for every retry defeats deduplication.

INTERVIEWER FOLLOW-UP

What if the original payment committed but the response was lost?

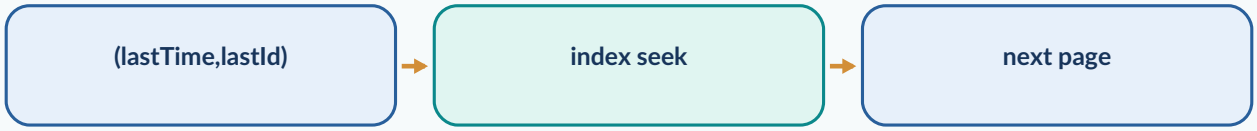
Answer: A retry with the same idempotency key must return the saved outcome or current state, not create another payment.

Why is cursor pagination often better than large offsets?

SHORT INTERVIEW ANSWER

A cursor seeks from a stable ordering key instead of scanning and discarding an ever-growing prefix. A deterministic tie-breaker prevents ambiguity when timestamps are equal.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
SELECT id, created_at FROM orders
WHERE customer_id = :customer
AND (created_at, id) < (:lastTime, :lastId)
ORDER BY created_at DESC, id DESC LIMIT 50;
```

WHY THIS WORKS

An index starting with customer_id and continuing with the ordering columns supports this access pattern. Sign or validate the cursor and bind it to the query scope. Concurrent edits can still change later pages unless the API promises a snapshot.

COMMON PITFALL A cursor is not an authorization credential; check ownership on every request.

INTERVIEWER FOLLOW-UP

What should happen when the sort key changes after page one?

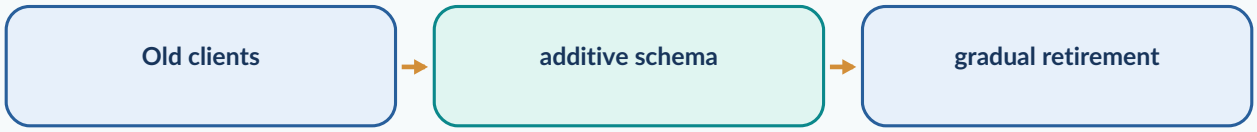
Answer: Define whether pages are a snapshot. If the ordering key can change, accept documented movement or use a stable snapshot cursor.

How do you evolve an API without breaking clients?

SHORT INTERVIEW ANSWER

Prefer additive compatible changes, explicit semantics and a transition period. Validate compatibility for serialized data and behavior, not just Java method signatures.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
record OrderView(String id, String status) {}
Later JSON may add estimatedDelivery.
Clients must tolerate unknown optional fields.
```

WHY THIS WORKS

Changing units, making a nullable value mandatory or reinterpreting a status can break clients even if JSON still parses. Define unknown enum behavior, deprecation telemetry and rollout order. For events, test old consumers against new producers and the reverse where required.

COMMON PITFALL Renaming a field without a compatibility strategy can break independently deployed services.

INTERVIEWER FOLLOW-UP

How will you know the last old client has migrated?

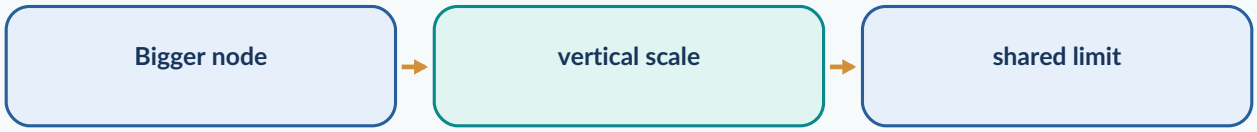
Answer: Measure old-client traffic through version headers, API telemetry and compatibility dashboards; retire only after an agreed cutoff.

How do vertical and horizontal scaling differ?

SHORT INTERVIEW ANSWER

Vertical scaling increases resources in one machine. Horizontal scaling adds machines and requires work and state to be distributed across them.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Vertical: a larger DB host reduces current CPU pressure.
Horizontal: four stateless Java API instances share load.
Shared bottleneck: one write-primary database.

WHY THIS WORKS

Vertical scaling can be a sensible first step with simpler operations. Horizontal scaling improves aggregate capacity only when shared resources and partitioning permit it. Plan what happens when one node fails and remaining nodes absorb its traffic.

COMMON PITFALL Doubling Java instances while keeping a saturated database unchanged can increase queueing.

INTERVIEWER FOLLOW-UP

Can the service meet its SLO after losing one instance?

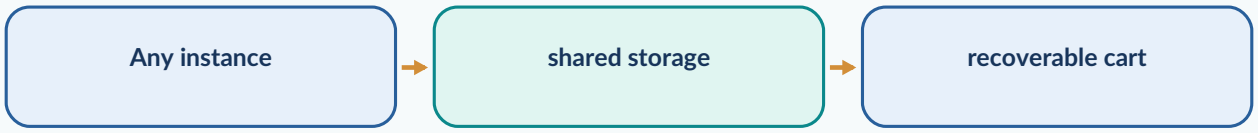
Answer: Test remaining-instance capacity using N-1 load testing. Keep SLO headroom, queues and downstream limits in the model.

What makes a service stateless?

SHORT INTERVIEW ANSWER

A stateless request handler does not require a particular instance's memory to process the next request. Durable business state and shared coordination live in appropriate external systems.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Bad scaling assumption:
Map<String, Cart> carts = new HashMap<>();
One instance's map is invisible to another instance.
```

WHY THIS WORKS

Local caches and temporary request data are compatible with a stateless architecture if correctness does not depend on them surviving or being shared. Store carts durably when the product promises recovery; avoid making sticky sessions the only recovery mechanism.

COMMON PITFALL Stateless does not mean that the business system has no state.

INTERVIEWER FOLLOW-UP

What happens to a cart when its instance restarts?

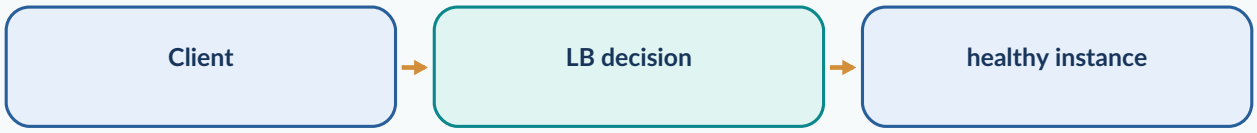
Answer: The cart survives only if stored in shared durable storage. An in-memory cart on one instance disappears on restart.

How do load-balancing algorithms affect a design?

SHORT INTERVIEW ANSWER

Round robin spreads requests by turn; least-connections considers current connection counts; hashing provides affinity. Pick a method that matches request duration, connection behavior and state needs.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Short HTTP calls: round robin can be adequate.
Long-lived sockets: connection-aware balancing helps.
Tenant affinity: hash routing may create hot tenants.

WHY THIS WORKS

A connection count is only a proxy for load: multiplexed connections or uneven request cost can make it misleading. Health checks and draining are as important as the selection algorithm. Weighted routing can support canaries or unequal capacity.

COMMON PITFALL A healthy process can still be unable to serve a critical dependency.

INTERVIEWER FOLLOW-UP

How will existing WebSocket connections move during deployment?

Answer: Open sockets usually remain on the old node until disconnected. Drain the node; clients reconnect using a last-seen event cursor.

What is the difference between a reverse proxy, gateway and service discovery?

SHORT INTERVIEW ANSWER

A reverse proxy routes requests to backends; a gateway often adds API policy such as authentication or quotas; service discovery finds the current addresses of service instances.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Gateway: validate token and route /orders.
Discovery: locate healthy order-service endpoints.
Java service: enforce order ownership and business rules.

WHY THIS WORKS

These responsibilities may overlap in products, but explaining them separately avoids misplaced guarantees. Discovery information can be stale, so callers still need bounded failures and endpoint refresh. Keep critical object authorization in the owning service even if the edge authenticates users.

COMMON PITFALL Putting every business rule in a gateway creates coupling and bypass risks.

INTERVIEWER FOLLOW-UP

What happens when discovery returns an instance that just terminated?

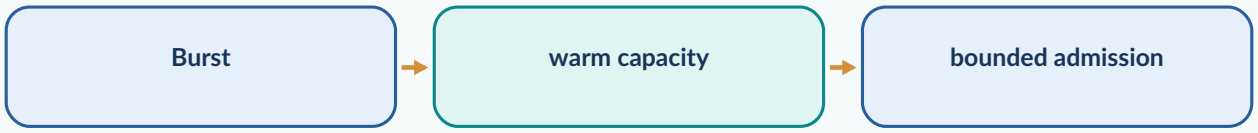
Answer: The call may fail or time out. Refresh discovery, retry safely within the deadline, and avoid sending to draining nodes.

How should autoscaling work under a traffic spike?

SHORT INTERVIEW ANSWER

Scale using a signal related to the bottleneck, keep warm capacity for rapid bursts, and bound queues while additional instances start. Scaling is part of admission control, not a substitute for it.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Queue wait rises before CPU saturates for blocking I/O.
Scale on suitable request/queue signals.
Reject excess work when safe capacity is exhausted.

WHY THIS WORKS

Startup time, cold caches and database connection limits can make aggressive scaling harmful. Add hysteresis and consider per-tenant fairness. For consumers, backlog age and sustainable processing rate can matter more than CPU.

COMMON PITFALL Creating many instances can create a simultaneous connection storm.

INTERVIEWER FOLLOW-UP

What protects the system during the minutes before new capacity is ready?

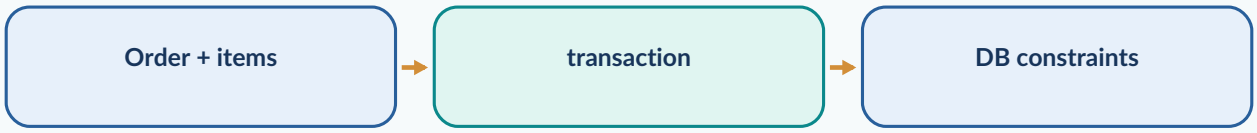
Answer: Protect with warm spare capacity, bounded queues, rate limits, graceful rejection and cached safe responses.

When should you use a relational database?

SHORT INTERVIEW ANSWER

Use it when relationships, constraints, transactions and flexible queries are central. Start with the required queries and invariants, then assess whether one database can support the workload.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

orders(id, customer_id, status, amount_minor)
order_items(order_id, product_id, quantity)
UNIQUE(customer_id, idempotency_key)

WHY THIS WORKS

A constraint protects the invariant across all application instances and scripts using that database. Relational systems can scale through indexing, replicas, partitioning and workload separation, with tradeoffs. A small service with relational data does not need a distributed database by default.

COMMON PITFALL Enforcing uniqueness with a Java read-then-insert check is vulnerable to races.

INTERVIEWER FOLLOW-UP

Which constraint prevents two retries creating different orders?

Answer: A unique index on (customer_id, idempotency_key), or the correct equivalent scope, rejects duplicate order creation.

When does a document or key-value store fit better?

SHORT INTERVIEW ANSWER

A document store can fit aggregate-oriented reads and changing nested structures; a key-value store fits direct lookup by key. Evaluate each product's actual consistency and transaction capabilities.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Document: product catalog with category-specific attributes.
Key-value: sessionId -> session metadata.
Query requirement: list products by price and brand.
```

WHY THIS WORKS

The third requirement still needs suitable indexing or a separate query model. Embedding data can improve reads while increasing update size and duplication. Avoid selecting a database solely because the schema changes; a relational system can also store structured JSON where appropriate.

COMMON PITFALL NoSQL does not mean no schema; the application still depends on data shape.

INTERVIEWER FOLLOW-UP

How will an update to embedded shared data propagate?

Answer: Update the shared source record and rebuild or notify affected documents; version copied fields to catch stale updates.

How do normalization and denormalization trade off?

SHORT INTERVIEW ANSWER

Normalization reduces duplication and update anomalies. Denormalization copies or precomputes data to serve a known read path faster, at the cost of keeping copies consistent.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Normalized: order references customer_id.
Historical snapshot: order stores shipping address at purchase.
Read projection: customer_order_summary stores counts.

WHY THIS WORKS

The address snapshot is part of historical business truth, not merely an optimization. A summary projection needs an update or rebuild strategy. Name which copy is authoritative and whether the user can tolerate projection lag.

COMMON PITFALL Denormalized fields without ownership and repair rules become inconsistent silently.

INTERVIEWER FOLLOW-UP

Can the summary be rebuilt from authoritative records?

Answer: Yes, if its source records and rules are retained. Replay or recompute it, then compare totals before switching reads.

How should indexes be designed for an API query?

SHORT INTERVIEW ANSWER

Design indexes around filters, ordering and selectivity, then verify with the database query plan and representative data. Indexes trade faster reads for storage and write maintenance.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Query: orders for one customer, newest first.
CREATE INDEX ix_order_customer_time
ON orders(customer_id, created_at DESC, id DESC);
```

WHY THIS WORKS

This index aligns with the cursor example. Whether it is used depends on data distribution, statistics and the full query. Explain covering/index-only possibilities separately from the basic lookup; do not assume every selected column should be indexed.

COMMON PITFALL An index on each column separately is not equivalent to a useful composite index.

INTERVIEWER FOLLOW-UP

How does this index affect a high-volume insert workload?

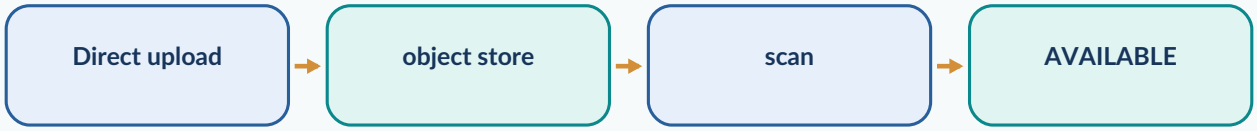
Answer: Every insert must also maintain this index, adding write I/O and storage. Measure before adding more indexes.

Why separate object storage from metadata storage?

SHORT INTERVIEW ANSWER

Large immutable files fit object storage, while ownership, status and queryable metadata fit a database. This avoids routing all file bytes through the relational database or application heap.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
DB: asset(id, owner_id, object_key, state, size_bytes)
Object store: binary bytes under a generated key.
State: UPLOADING -> SCANNING -> AVAILABLE.
```

WHY THIS WORKS

Access should depend on the metadata state and authorization. Because object storage and the database are different systems, handle orphaned objects, missing objects and interrupted uploads with cleanup and reconciliation.

COMMON PITFALL A successful byte upload does not automatically mean the asset is safe to publish.

INTERVIEWER FOLLOW-UP

Which job detects uploaded files whose finalization never completed?

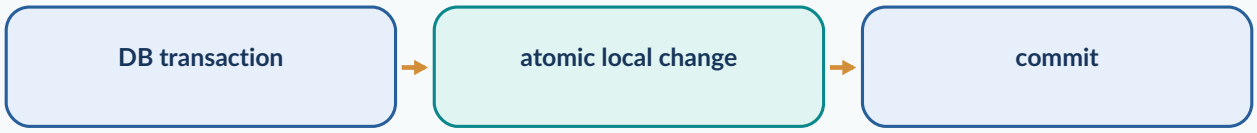
Answer: A scheduled reconciler searches uploads stuck in UPLOADING/SCANNING, verifies object state and completes or cleans them.

What do ACID properties guarantee?

SHORT INTERVIEW ANSWER

Atomicity groups operations, consistency means declared invariants remain valid through correct transactions, isolation controls concurrent interaction, and durability concerns persistence of committed results under the system's failure model.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Transfer within one database:
BEGIN; debit account A; credit account B; COMMIT;
Both balance changes commit together or neither does.
```

WHY THIS WORKS

Application logic and constraints are still responsible for business correctness. Durability depends on the database and storage configuration. A local database transaction does not automatically make an external payment provider part of the same commit.

COMMON PITFALL ACID consistency is different from distributed read consistency such as linearizability.

INTERVIEWER FOLLOW-UP

What happens if an HTTP call succeeds before the DB transaction rolls back?

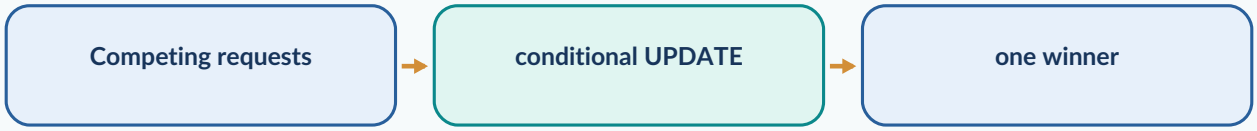
Answer: The outside call is not rolled back with the database. Record uncertainty and compensate or reconcile safely.

How can two requests safely compete for the last stock unit?

SHORT INTERVIEW ANSWER

Use an atomic conditional update and check the affected-row count, or use an appropriate locking transaction. Enforce the stock change and related reservation in one database transaction.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
UPDATE inventory
SET available = available - :qty
WHERE product_id = :id AND available >= :qty;
// require qty > 0; success means exactly one row changed
```

WHY THIS WORKS

A positive quantity check is essential. The database arbitrates concurrent updates; zero rows means missing product or insufficient stock and may require a defined response. Record a stable reservation key in the same transaction to handle duplicate commands.

COMMON PITFALL A Java synchronized block protects only cooperating threads in one JVM.

INTERVIEWER FOLLOW-UP

Where will the reservation record be committed relative to the decrement?

Answer: Insert the reservation in the same transaction as the conditional decrement; commit both or neither.

How do optimistic and pessimistic locking differ?

SHORT INTERVIEW ANSWER

Optimistic locking detects conflicting versions at update time. Pessimistic locking reserves access using database locks before making a decision. Choose based on contention and transaction length.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
UPDATE orders SET status = :next, version = version + 1
WHERE id = :id AND version = :expected;
// zero rows: stale version or missing record
```

WHY THIS WORKS

Optimistic conflicts need a business-aware response; do not blindly retry decisions computed from stale data. Pessimistic locks can serialize contention but increase waiting and deadlock risk. Keep locked transactions short and acquire multiple locks in a consistent order.

COMMON PITFALL Version checks protect only writes that participate in the protocol.

INTERVIEWER FOLLOW-UP

Should a stale address edit be retried or shown as a conflict?

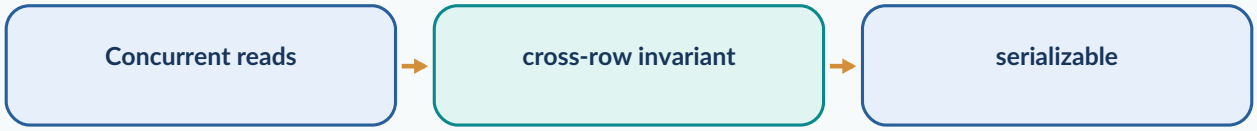
Answer: Show a 409/conflict and ask the user to review changes; automatic retry could overwrite another user's edit.

Why can isolation level change the result of a business operation?

SHORT INTERVIEW ANSWER

Isolation levels allow different concurrent histories. A read followed by a decision can become invalid before commit unless the operation uses sufficient isolation, locks or constraints.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Two doctors both read: another doctor is on call.
Each turns their own on_call flag off.
Row-version checks alone can still leave no doctor on call.

WHY THIS WORKS

This write-skew example touches different rows, so locking only the modified row is insufficient. Define the cross-row invariant and protect it with a suitable serializable transaction or an explicit coordination row. Database isolation names and guarantees vary by product.

COMMON PITFALL PostgreSQL READ COMMITTED gives a new snapshot per statement, not one immutable transaction snapshot.

INTERVIEWER FOLLOW-UP

What must be retried after a serialization failure?

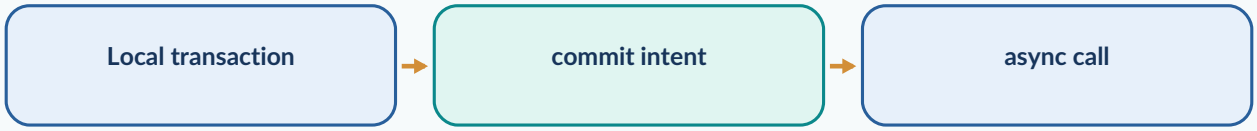
Answer: Retry the entire transaction from a fresh read, with a bounded retry budget, not just its last SQL statement.

How should Java services handle transaction boundaries?

SHORT INTERVIEW ANSWER

Place the boundary around a complete local business operation, keep it short, and understand the framework's interception rules. Avoid holding locks while waiting for slow remote work.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
@Transactional
public void reserve(Command c) {
    inventory.decrementIfAvailable(c);
    reservations.insert(c.requestId());
}
```

WHY THIS WORKS

This Spring fragment assumes a transactional repository and an intercepted call. In default proxy mode, self-invocation does not trigger transaction advice. Verify rollback rules for the exception types you use. External calls belong in an explicit workflow with compensation or reconciliation.

COMMON PITFALL Returning successfully after catching a business failure may accidentally commit partial intended work.

INTERVIEWER FOLLOW-UP

How would you verify both rows roll back after an insert failure?

Answer: Force the reservation insert to fail in an integration test, then assert stock and reservation rows are unchanged.

What is the difference between replication and sharding?

SHORT INTERVIEW ANSWER

Replication stores copies of the same data on multiple nodes. Sharding distributes different subsets across nodes. Each shard may have its own replicas.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Shard A: customers 1-999, replicated three times.
Shard B: customers 1000-1999, replicated three times.
Replication changes copies; sharding changes placement.
```

WHY THIS WORKS

Replication can improve read capacity and fault recovery but does not remove the primary's write workload in a single-leader design. Sharding spreads writes but complicates cross-shard queries, rebalancing and transactions. Begin with partitioning only when workload or ownership requires it.

COMMON PITFALL A replica and a shard are not interchangeable terms.

INTERVIEWER FOLLOW-UP

How will a customer query route to the correct shard?

Answer: Hash or look up customer_id in the shard map; version routing so moves do not send writes to two owners.

Why can reading from a replica return stale data?

SHORT INTERVIEW ANSWER

Asynchronous replication may not have applied the primary's latest commit. A successful write on the primary does not mean every replica can immediately return it.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
User changes profile name and receives success.
Next GET is routed to a lagging replica.
Response briefly contains the old name.
```

WHY THIS WORKS

Possible policies include reading the primary for a session after a write, waiting for a relevant replication position, or explicitly accepting stale data. Define read-your-writes expectations for each operation. A routing timeout or failover can complicate a simple stickiness policy.

COMMON PITFALL A healthy replica is not necessarily a sufficiently fresh replica.

INTERVIEWER FOLLOW-UP

Which metric prevents reads from replicas with unacceptable lag?

Answer: Monitor replication lag in time or log position, and route consistency-sensitive reads to primary when lag exceeds policy.

How do you choose a partition key?

SHORT INTERVIEW ANSWER

Choose a key that distributes load while keeping common operations within one partition. Consider traffic distribution, data size, ordering and future tenant growth.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Orders partitioned by customer_id:
customer history is local; global sales reports are not.
Events partitioned by order_id:
per-order event order is easier to preserve.
```

WHY THIS WORKS

The right key differs for storage and an event log. A tenant key simplifies isolation but a large tenant may dominate one partition. Hashing improves distribution but range scans may need fan-out or a separate index.

COMMON PITFALL Even distribution of record counts does not guarantee even distribution of traffic.

INTERVIEWER FOLLOW-UP

What happens when one tenant grows one hundred times larger?

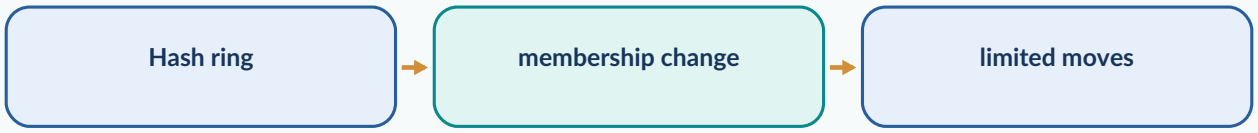
Answer: Split the tenant by a secondary key if feasible, dedicate it to a shard, or isolate its workload; preserve required ordering.

What does consistent hashing solve?

SHORT INTERVIEW ANSWER

It maps keys to nodes so membership changes move a limited subset of keys instead of remapping most keys as simple modulo hashing can. It does not solve storage migration by itself.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Modulo routing: hash(key) % nodeCount.
Changing 4 nodes to 5 changes many destinations.
Hash ring: keys move mainly near the changed ownership range.

WHY THIS WORKS

Virtual nodes can smooth uneven placement. Rendezvous hashing is another approach: choose the node with the highest key-node score, as demonstrated in the Java lab. Real changes require data movement, versioned routing and handling old owners during transition.

COMMON PITFALL Calling the algorithm consistent does not imply strong read consistency.

INTERVIEWER FOLLOW-UP

How will reads work while a range is being copied?

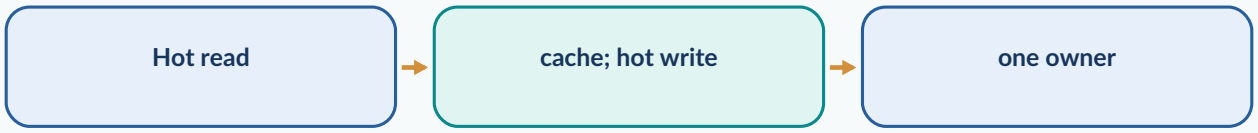
Answer: Use a migration-aware router: check old and new owners during transition with versioned ownership and cutover rules.

How do you handle a hot key or a hot partition?

SHORT INTERVIEW ANSWER

Measure whether the heat is reads, writes or fan-out, then use a mitigation that preserves the required semantics. Options include replication, caching, batching or partitioning the work differently.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Popular product reads: CDN/cache and local read copies.
Popular product stock: one authoritative reservation policy.
Analytics counter: split increments and later aggregate.

WHY THIS WORKS

A split counter is acceptable when an approximate or delayed total is sufficient. Splitting stock into uncontrolled independent counters can oversell. Hot-key mitigation often sacrifices immediacy, simplicity or exactness; state that cost explicitly.

COMMON PITFALL Adding more shards does not help a key that still belongs to one shard.

INTERVIEWER FOLLOW-UP

Which operations can safely use a delayed aggregate?

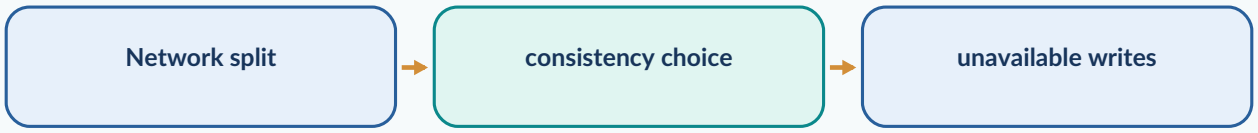
Answer: Dashboards, view counts and noncritical analytics may use delayed sums. Payment, exact stock and seat ownership may not.

What does the CAP theorem actually say?

SHORT INTERVIEW ANSWER

During a network partition, a distributed read/write system cannot guarantee both linearizable consistency and a successful response to every request reaching a non-failing node under the theorem's model.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Two isolated replicas receive conflicting stock updates.
Option: reject some writes to protect one ordering.
Option: accept writes and later resolve divergence.

WHY THIS WORKS

Partition tolerance is not a feature you can disable while networks still fail. Outside a partition, systems still make latency and coordination tradeoffs. Discuss the operation and its failure behavior rather than branding a whole application as CP or AP.

COMMON PITFALL CAP availability is not the same as a 99.9% business success SLO.

INTERVIEWER FOLLOW-UP

Can stale catalog reads continue while stock writes are rejected?

Answer: Yes. Catalog viewing can tolerate staleness, but stock writes may be denied until one authority is available.

How do strong, eventual and read-your-writes consistency differ?

SHORT INTERVIEW ANSWER

Linearizable operations behave as if they occur in one real-time order. Eventual consistency promises convergence under its stated assumptions. Read-your-writes ensures a session sees its own acknowledged updates.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Stock confirmation: require an authoritative decision.
Search index: accept documented update lag.
Profile edit: preserve the editor's read-your-writes experience.

WHY THIS WORKS

These guarantees are not synonyms for isolation levels. Specify convergence conditions, allowed staleness and what happens during failover. Session guarantees may require tokens or routing; they should not depend on accidental connection affinity.

COMMON PITFALL Eventual consistency alone does not specify how many seconds stale data may remain.

INTERVIEWER FOLLOW-UP

What happens if updates stop but a replica cannot reconnect?

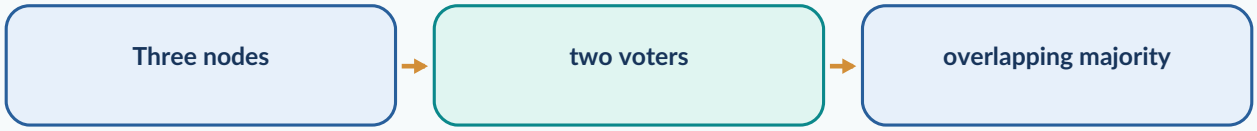
Answer: It will remain behind until it reconnects or is rebuilt. Eventual consistency does not promise progress during permanent isolation.

Why is a majority quorum useful, and what are its limits?

SHORT INTERVIEW ANSWER

Overlapping majorities can support coordinated decisions when used by a correct protocol. A simple read/write quorum arithmetic rule alone does not establish linearizability.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
N = 3; majority = 2.
Any two majorities intersect.
R + W > N is an overlap condition, not a full protocol.
```

WHY THIS WORKS

Version handling, concurrent writes, membership changes, failures and read paths matter. Sloppy quorums and asynchronous application can alter guarantees. When a design requires consensus, use a proven implementation and explain what losing quorum means for operations.

COMMON PITFALL Reading two arbitrary replicas is not automatically a linearizable read.

INTERVIEWER FOLLOW-UP

Can a minority serve safe writes while disconnected from the majority?

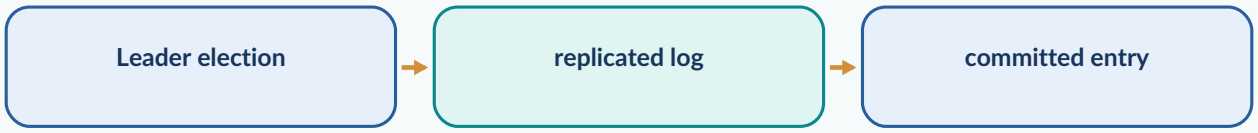
Answer: Not as quorum-committed writes. It can reject or queue requests; accepting them as final risks divergence.

What problem does Raft solve?

SHORT INTERVIEW ANSWER

Raft coordinates a replicated log through leader election, log replication and safety rules. It lets a replicated state machine agree on committed operations under its fault assumptions.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Three voting nodes tolerate one unavailable node.
Five voting nodes can tolerate two unavailable nodes.
No majority: progress may stop, while safety is preserved.

WHY THIS WORKS

Raft is not a replacement for business validation, partitioning or disaster recovery. Followers may be behind; safe reads require the implementation's read protocol. Leadership alone does not authorize arbitrary local writes without commitment rules.

COMMON PITFALL Do not claim that electing a leader makes every follower read current.

INTERVIEWER FOLLOW-UP

Why does a three-node cluster need two votes to make progress?

Answer: A majority of 3 is 2; any two sets of 2 nodes overlap, helping preserve one committed log history.

Why do distributed locks need ownership checks and sometimes fencing?

SHORT INTERVIEW ANSWER

A lease can expire while an old holder is paused. Ownership-safe release prevents deleting another holder's lease, while fencing lets the protected resource reject stale owners' writes.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Holder A gets fence 7, then pauses.
Holder B gets fence 8 and writes.
Resource rejects A's later write carrying fence 7.

WHY THIS WORKS

The resource must durably enforce monotonic fencing; adding a token without checking it does nothing. A lock service and the database can fail independently. Prefer a local database constraint or conditional transaction when it directly enforces the invariant.

COMMON PITFALL A TTL does not guarantee the old holder stops executing when its lease expires.

INTERVIEWER FOLLOW-UP

Where is the last accepted fencing token stored?

Answer: The resource (often a database row) stores the highest accepted token and rejects writes with smaller tokens.

How do cache-aside, write-through and write-behind differ?

SHORT INTERVIEW ANSWER

Cache-aside loads on a miss and writes authoritative storage separately. Write-through includes cache updates in the write path. Write-behind defers persistence and therefore changes the durability contract.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Cache-aside Java sketch:
var cached = cache.get(key);
if (cached != null) return cached;
var value = repository.find(key);
cache.put(key, value); return value;
```

WHY THIS WORKS

This fragment omits TTL, concurrent misses and invalidation intentionally. Write-through across two independent systems still needs failure handling; it is not automatically atomic. Write-behind is suitable only when acknowledged data-loss and recovery risks are acceptable or durably queued.

COMMON PITFALL A successful cache write is not proof that a durable database write succeeded.

INTERVIEWER FOLLOW-UP

Which store remains authoritative after a cache outage?

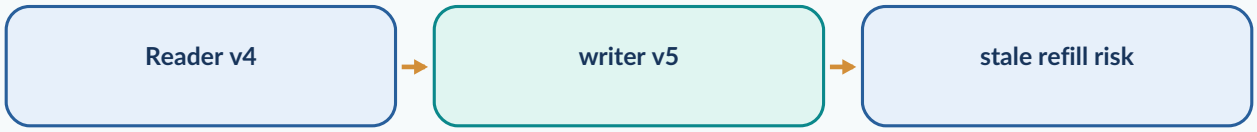
Answer: The durable database stays authoritative in cache-aside. A cache outage must not silently become lost data.

Why is cache invalidation hard?

SHORT INTERVIEW ANSWER

A read can race with a write and refill old data after invalidation. Correctness depends on update ordering, versions, retention and the consistency the API promises.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Reader loads version 4 from DB.
Writer commits version 5 and deletes cache entry.
Reader then places version 4 back into the cache.

WHY THIS WORKS

A TTL limits how long such data may remain only if the refill and expiry semantics are controlled; it does not prevent the race. Version-aware writes, change events and bounded stale policies can help. For correctness-critical decisions, consult authoritative state rather than trusting a cache.

COMMON PITFALL Deleting after a database write alone does not eliminate all stale refill races.

INTERVIEWER FOLLOW-UP

Can the cache reject a lower-version replacement?

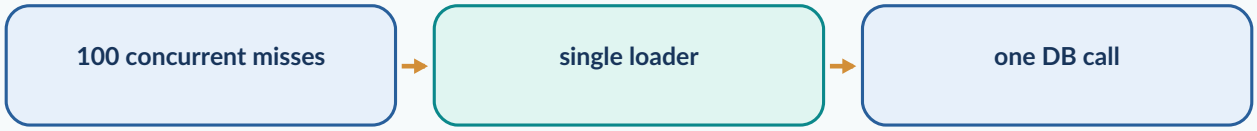
Answer: Yes: attach a version and update only if the incoming version is not older than the cached version.

What causes a cache stampede and how do you prevent it?

SHORT INTERVIEW ANSWER

A popular key expires or disappears and many callers reload it simultaneously. Coalesce misses, spread expiry times, warm critical data or serve permitted stale values while one refresh runs.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
100 callers miss product:42.
Single-flight: one loader; 99 wait for its result.
TTL jitter: avoid thousands of unrelated keys expiring together.
```

WHY THIS WORKS

Single-flight in one JVM still allows one load per application instance. A distributed refresh policy adds lease and failure complexity. Bound waiting and loading time; a stuck loader should not block the key indefinitely. Keep stale serving separate from availability of authoritative decisions.

COMMON PITFALL Jitter alone does not prevent a stampede on one extremely hot key.

INTERVIEWER FOLLOW-UP

What happens to waiters if the loader throws an exception?

Answer: Fail or time out the shared load, release waiting callers with an error/fallback and permit a later bounded retry.

How do TTL and eviction differ?

SHORT INTERVIEW ANSWER

TTL makes an entry logically expire after a time policy. Eviction removes entries to manage capacity. A cache hit policy must account for both and tolerate entries disappearing early.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

TTL: product data is fresh for 60 seconds.
Eviction: memory pressure removes it after 5 seconds.
Next request reloads even though TTL was not reached.

WHY THIS WORKS

Choose an eviction strategy based on access patterns and budget memory for implementation overhead. A Redis cache can use approximate LRU or LFU policies; an authoritative Redis workload requires a different persistence and eviction discussion. Cache size should cover the useful working set, not necessarily the whole database.

COMMON PITFALL TTL is not a guarantee that a key will remain resident for that duration.

INTERVIEWER FOLLOW-UP

How will you detect cache churn that overloads the database?

Answer: Track miss ratio, eviction rate, hot-key reloads and database queries. A sharp miss/load rise signals churn.

When should you use a CDN and HTTP caching?

SHORT INTERVIEW ANSWER

Use edge caching for cacheable content that many users request, with explicit freshness, validators and privacy rules. It reduces origin traffic and often improves geographic latency.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Public asset: Cache-Control: public, max-age=86400.
Versioned path: /assets/app.a83f.js.
Private account response: use an appropriate private/no-store policy.

WHY THIS WORKS

An ETag can support conditional validation. Shared-cache keys must include every relevant response variant; authenticated or personalized content needs careful policy. Purges and versioned URLs have different costs. Cache directives do not replace access control at the origin.

COMMON PITFALL A cache key missing tenant or language context can return the wrong user's variant.

INTERVIEWER FOLLOW-UP

What happens when a deleted public file is still cached at an edge?

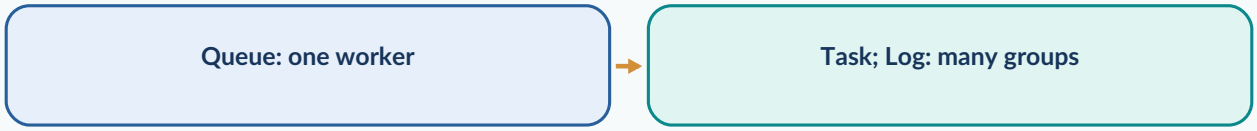
Answer: The edge may keep serving it until expiry or purge. Use purge/versioning and choose cache TTL to match revocation needs.

What is the difference between a work queue and an event log?

SHORT INTERVIEW ANSWER

A work queue commonly distributes tasks among workers. A retained event log supports replay and independent consumer positions. Product capabilities overlap, so describe the required behavior.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Queue use: send one email task to a worker.
Log use: OrderCreated consumed by billing and analytics.
Each log consumer group tracks its own progress.

WHY THIS WORKS

A log can also distribute work within a group, while some queues retain history. Replay changes the operational contract: old events must remain interpretable and side effects must not be repeated accidentally. Decide retention, acknowledgement and poison-message handling.

COMMON PITFALL A message broker is not automatically a permanent source of business truth.

INTERVIEWER FOLLOW-UP

Can a new analytics consumer rebuild six months of history?

Answer: Only if the log still retains that data, or a snapshot/archive is available. Six months is not guaranteed by default.

How do at-most-once and at-least-once delivery differ?

SHORT INTERVIEW ANSWER

At-most-once can lose work but avoids repeat delivery under its contract. At-least-once can redeliver work, so effects need deduplication or natural idempotency.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Consumer updates DB, then crashes before acknowledging.
Broker redelivers the event after recovery.
Consumer must avoid applying the business effect twice.

WHY THIS WORKS

Acknowledging before an effect risks loss; acknowledging afterward risks duplication. A unique event key stored in the same database transaction as the effect can close the local duplication window. It does not make a remote side effect atomic with that transaction.

COMMON PITFALL Duplicate transport delivery and duplicate business effect are different measurements.

INTERVIEWER FOLLOW-UP

Where is the dedup marker committed relative to the effect?

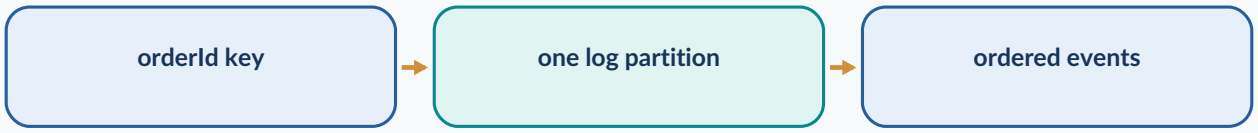
Answer: Commit the dedup marker and local business update in the SAME database transaction so neither can happen alone.

What ordering can a partitioned log provide?

SHORT INTERVIEW ANSWER

A partition can maintain a sequence of records. Ordering across partitions or across independently produced operations requires additional coordination or explicit business versioning.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Kafka key = orderId.
Events for one key normally route to one partition.
Different orders may be processed concurrently.

WHY THIS WORKS

Producer configuration and concurrent producers also affect the business order represented by the log. Consumers that dispatch to parallel workers can complete effects out of order even when delivery order is stable. Sequence numbers can detect gaps and obsolete updates.

COMMON PITFALL Partition delivery order does not guarantee ordered completion in a multithreaded consumer.

INTERVIEWER FOLLOW-UP

How should a consumer handle version 8 arriving after version 9?

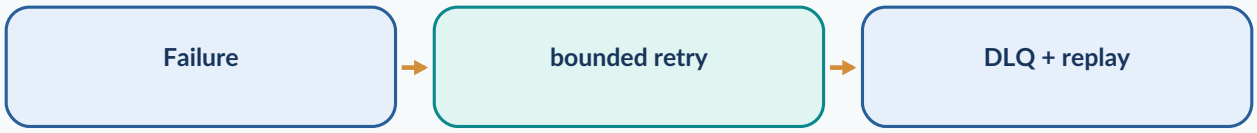
Answer: Ignore older version 8 after 9, or repair a detected gap; do not apply state transitions backward.

How should retries and dead-letter handling work for consumers?

SHORT INTERVIEW ANSWER

Retry transient failures within a bounded policy, preserve the needed order, and isolate permanently failing records with enough context for investigation and safe replay.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Event e-17 fails validation permanently.
Record event ID, schema version and reason.
Move to a controlled failure workflow; do not retry forever.
```

WHY THIS WORKS

A retry topic can let later events overtake earlier ones. If an aggregate requires strict order, pause or quarantine that aggregate rather than silently skipping its state transition. A dead-letter store needs owners, alerts, retention and a replay procedure.

COMMON PITFALL A dead-letter queue without monitoring turns visible failures into hidden backlog.

INTERVIEWER FOLLOW-UP

Can replay repeat a notification or payment already completed?

Answer: Yes. Replay may resend unless effects are idempotent or tracked using durable external-operation IDs.

What does backpressure mean in a streaming system?

SHORT INTERVIEW ANSWER

Downstream limits should influence upstream admission or production so slow processing does not create unbounded memory and latency. When slowing producers is impossible, define shedding or durable buffering.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Consumer capacity: 500 events/s.
Producer rate: 700 events/s.
Backlog grows by 200 events/s while this imbalance continues.

WHY THIS WORKS

Measure oldest-message age as well as count. A durable log buys time, but finite retention and storage still impose limits. Parallelism helps only if partitioning, downstream capacity and ordering allow it. Treat replay throughput as a separate workload competing with live processing.

COMMON PITFALL A larger queue postpones overload; it does not increase processing rate.

INTERVIEWER FOLLOW-UP

How long until retention is exceeded at the current growth rate?

Answer: Backlog growth is 200 events/s in the example. Time to exceed capacity = free retained slots / 200 seconds.

What problem does the transactional outbox solve?

SHORT INTERVIEW ANSWER

It records the business change and an event to publish in one local database transaction, avoiding the gap between committing data and separately publishing an event.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
BEGIN;
INSERT INTO orders(...);
INSERT INTO outbox(event_id, aggregate_id, payload) VALUES(...);
COMMIT;
```

WHY THIS WORKS

A relay or change-data-capture process publishes committed outbox rows. If publication succeeds but marking completion fails, it may publish again. Consumers therefore still need duplicate handling. Track oldest unpublished age and clean up only according to recovery and retention policy.

COMMON PITFALL Outbox provides durable publish intent, not automatically exactly-once delivery.

INTERVIEWER FOLLOW-UP

What happens if the relay crashes immediately after publishing?

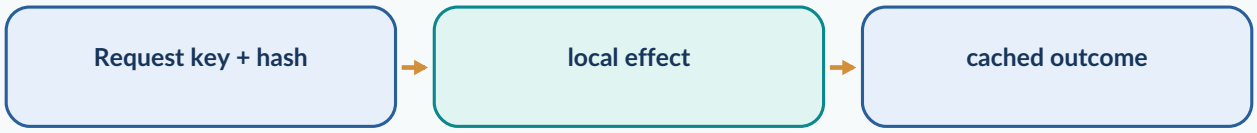
Answer: It publishes again after restart because the relay did not mark success. Consumers must ignore duplicate event IDs.

How should an idempotency key be implemented?

SHORT INTERVIEW ANSWER

Scope the key to a caller and operation, bind it to a canonical validated request, atomically coordinate ownership and store a durable result or status. Define expiry and duplicate-in-progress behavior.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
UNIQUE(caller_id, operation, request_key)
Fields: request_hash, state, result_reference.
Same key + different request hash -> conflict.
```

WHY THIS WORKS

Canonicalization must handle semantically equivalent input consistently. A local database operation can commit the key and effect together. For an external effect, persist workflow state and use the provider's stable key plus reconciliation. Never claim an in-memory map is a durable multi-instance idempotency store.

COMMON PITFALL Expiring keys too early can turn a delayed retry into a second operation.

INTERVIEWER FOLLOW-UP

What response should a concurrent duplicate receive while the first request is running?

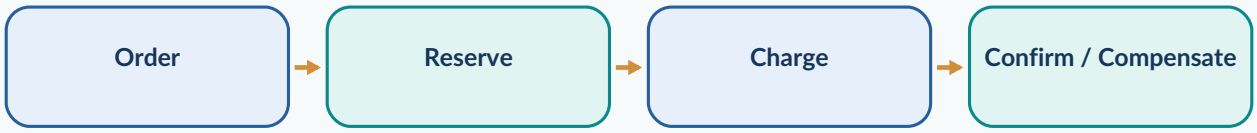
Answer: Return the existing result if finished; otherwise respond with an in-progress status or wait briefly and safely.

What is a saga, and when is it useful?

SHORT INTERVIEW ANSWER

A saga coordinates local transactions across services using explicit steps and compensating actions. It supports recoverable workflows when one global atomic transaction is unavailable or undesirable.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Reserve inventory -> authorize payment -> confirm order.
If authorization fails: release reservation.
If later work fails: retry, reconcile or compensate by policy.
```

WHY THIS WORKS

Compensation is a business operation, not a database rollback of the past. It can fail and needs idempotency, retry and audit. Orchestration centralizes workflow state; choreography reacts to events but can make the overall process harder to inspect.

COMMON PITFALL A payment refund may have different timing and rules from undoing an uncommitted payment.

INTERVIEWER FOLLOW-UP

Who owns a workflow stuck between authorization and confirmation?

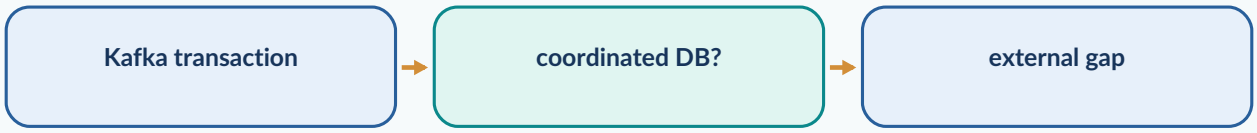
Answer: A durable workflow owner (saga coordinator/order service) stores state and runs recovery/reconciliation jobs.

Why is exactly-once processing a bounded claim?

SHORT INTERVIEW ANSWER

It describes a guarantee within a specified system and protocol. It does not automatically cover arbitrary external effects, replay, client retries and all failure modes.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Kafka transaction: atomically write Kafka output and offsets.
HTTP email provider: outside that transaction.
DB unique event key: can protect one local DB effect.

WHY THIS WORKS

Name the boundary, isolation, producer/consumer configuration and effect. Producer idempotence handles certain duplicate writes to Kafka; it is different from an end-to-end payment guarantee. A design can often achieve effectively-once business behavior with stable identities and reconciliation.

COMMON PITFALL Do not label the entire workflow exactly-once because one broker supports transactions.

INTERVIEWER FOLLOW-UP

Which side effect remains outside the coordinated commit?

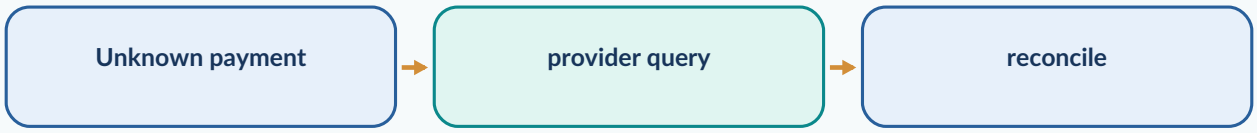
Answer: Any provider call outside the coordinated database/log transaction, such as an email or payment API, can repeat.

How do reconciliation and audit improve reliability?

SHORT INTERVIEW ANSWER

Reconciliation compares durable records across boundaries and repairs or escalates mismatches. Audit explains what commands and transitions occurred without relying on temporary process memory.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Local payment state = PENDING, provider state = SUCCEEDED.
Reconciler queries using the original provider reference.
It advances the workflow without charging again.

WHY THIS WORKS

Run reconciliation on a schedule and after incident recovery. Use bounded scans or cursors, idempotent repairs and an escalation state for ambiguous cases. An audit log should capture stable IDs and decisions while avoiding secrets and unnecessary personal data.

COMMON PITFALL Treating every unknown external outcome as failure can create duplicate effects on retry.

INTERVIEWER FOLLOW-UP

What evidence distinguishes a failed charge from a lost success response?

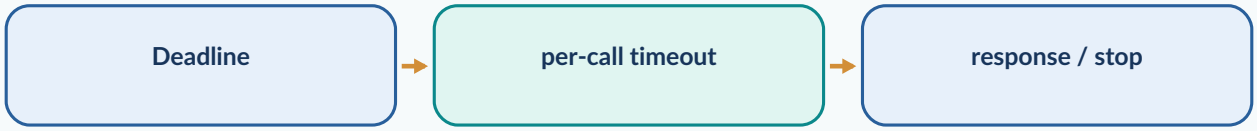
Answer: Check the provider using its stable payment reference and webhook/status API; a timeout alone proves neither success nor failure.

How should you set timeouts and end-to-end deadlines?

SHORT INTERVIEW ANSWER

Bound each stage and carry a remaining deadline through the request path. Include connection acquisition and queue waiting rather than budgeting only the remote response time.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Client deadline: 800 ms.
Reserve 100 ms for local processing and response.
Calls, pool wait and permitted retries share the remainder.

WHY THIS WORKS

Choose budgets from observed latency, the user objective and acceptable false timeouts, then load-test them. A timeout means the caller stopped waiting; it does not establish that the callee stopped work or rolled back. Cancellation must be cooperative and propagated where supported.

COMMON PITFALL Giving every nested call the full original deadline can exceed the user's budget.

INTERVIEWER FOLLOW-UP

How much time remains before starting the second attempt?

Answer: Subtract elapsed time and reserved safety budget from the overall deadline; skip retries with too little time left.

When should you retry, and why add jitter?

SHORT INTERVIEW ANSWER

Retry operations whose failures may be transient and whose effects are safe to repeat, within a bounded attempt and time budget. Randomized delay reduces synchronized retry waves.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Delay cap = min(maxDelay, base * 2^attempt).
Full jitter example: random delay from 0 to that cap.
Do not retry invalid input or an exhausted total deadline.

WHY THIS WORKS

Classify errors using the API contract. Retry at one deliberate layer to avoid multiplicative attempts. A payment timeout requires an idempotency or status lookup contract, not an optimistic assumption of failure. Respect relevant rate-limit signals and avoid retrying into a saturated dependency.

COMMON PITFALL Three attempts at each of three nested layers can create 27 downstream attempts.

INTERVIEWER FOLLOW-UP

Which layer owns retry policy for this business operation?

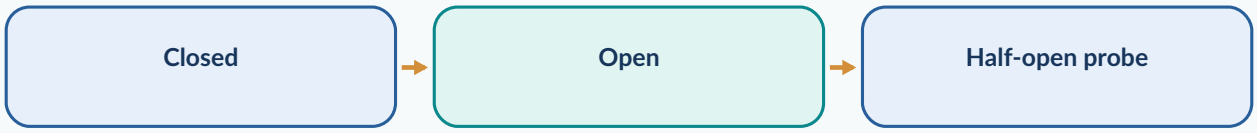
Answer: Choose one responsible layer for retries; prevent multiplied attempts across client, gateway and each service.

How does a circuit breaker work?

SHORT INTERVIEW ANSWER

It stops most calls after a failure policy is triggered, then allows controlled probes to see whether the dependency recovered. It limits repeated failures rather than fixing the dependency.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Closed: normal calls, collect outcomes.
Open: fail fast or use a defined fallback.
Half-open: limited probes before normal traffic returns.

WHY THIS WORKS

Choose whether timeouts, selected errors and slow calls count as failures. Keep breaker scope aligned with the dependency or operation; one broken tenant should not necessarily trip every tenant. Fallback data must still satisfy the user contract.

COMMON PITFALL Circuit breakers do not bound the number of successful but slow concurrent calls; bulkheads can.

INTERVIEWER FOLLOW-UP

What happens if all instances enter half-open together?

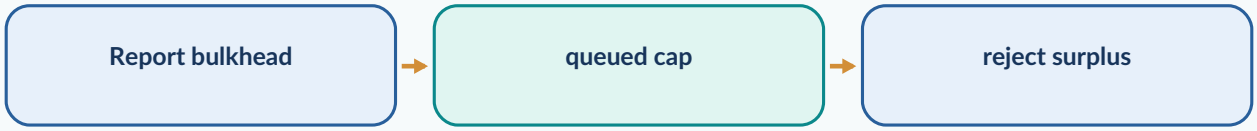
Answer: Use jitter, per-instance probe caps and half-open trial limits so all nodes do not hit recovery at once.

What is a bulkhead, and how is it different from rate limiting?

SHORT INTERVIEW ANSWER

A bulkhead limits concurrent work or isolates resources so one dependency cannot consume the whole service. Rate limiting controls admission over time; the two address different overload shapes.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Payment calls: semaphore permits = 20.
Reports: separate bounded executor and DB budget.
Request quota: 100 calls per user per minute.

WHY THIS WORKS

A low request rate can still exhaust concurrency if each call hangs. Acquire with a bounded wait or fail fast; release permits in finally blocks. Separate resource pools only where the isolation benefit justifies the extra overhead.

COMMON PITFALL A semaphore does not enforce a fixed requests-per-second quota.

INTERVIEWER FOLLOW-UP

What happens when a report request blocks for several minutes?

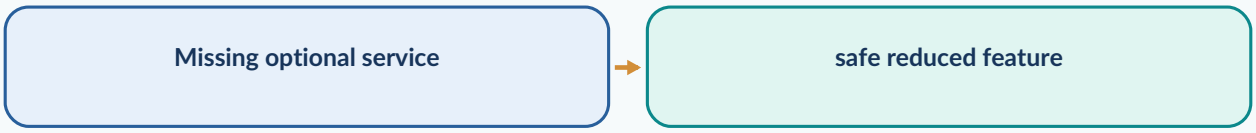
Answer: Put reports in a separate bounded executor. When full, reject the new report rather than block checkout threads.

What is graceful degradation?

SHORT INTERVIEW ANSWER

It deliberately reduces optional behavior while preserving essential correctness and a clear user experience. The fallback must be explicitly allowed by the product's requirements.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Catalog page: omit recommendations during an outage.
Stock purchase: reject or remain pending if authority is unavailable.
Search: serve a documented stale index when acceptable.

WHY THIS WORKS

Define which dependencies are optional, which failure modes trigger fallback and how recovery happens. Monitor degraded responses separately from fully successful ones. Avoid returning fabricated confirmation for a write whose outcome is unknown.

COMMON PITFALL A stale price or authorization decision may violate the contract even when it improves uptime.

INTERVIEWER FOLLOW-UP

Which user journey can continue safely without this dependency?

Answer: Keep login, read-only catalog or previously confirmed order lookup if safe; block uncertain payment or inventory writes.

How should you size a Java thread pool?

SHORT INTERVIEW ANSWER

Start with workload type and resource limits, then measure throughput and latency. CPU-bound work is constrained by effective CPU capacity; blocking I/O also depends on downstream capacity and wait time.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
var pool = new ThreadPoolExecutor(
    8, 8, 0, TimeUnit.SECONDS,
    new ArrayBlockingQueue<Runnable>(200),
    new ThreadPoolExecutor.AbortPolicy());
```

WHY THIS WORKS

This bounded example caps execution and queued tasks; it is not a universal recommendation of eight threads. Rejection must become a controlled API outcome. Common fixed-thread-pool factory methods can use unbounded queues, so inspect the actual executor. Count queued work in the deadline.

COMMON PITFALL A huge queue can preserve acceptance while making every result too late to be useful.

INTERVIEWER FOLLOW-UP

What happens when the 201st waiting task arrives?

Answer: If all workers and queue slots are occupied, reject or apply a documented backpressure policy; never grow without bound.

Do virtual threads remove the need for concurrency limits?

SHORT INTERVIEW ANSWER

No. Java 21+ virtual threads make many blocking tasks cheaper in thread resources, but database connections, remote quotas, CPU and memory still need explicit limits.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Java 21+ concept:
Executors.newVirtualThreadPerTaskExecutor()
Still bound payment calls and database access separately.

WHY THIS WORKS

Use virtual threads for suitable blocking workloads, then observe actual behavior on the target JDK and libraries. They do not make CPU-heavy work faster merely by adding concurrency. The runnable exercises in this guide use Java 17 so they require no preview flags or newer runtime.

COMMON PITFALL Unlimited cheap threads can still create unlimited expensive downstream work.

INTERVIEWER FOLLOW-UP

Where will you enforce the database concurrency budget?

Answer: Use a semaphore or pool limit before DB calls. Virtual threads do not create extra DB connections.

Why must database connection pools be budgeted across instances?

SHORT INTERVIEW ANSWER

Each application instance adds potential database sessions. The total must fit the database's sustainable concurrency and leave room for administration, background jobs and other applications.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
12 instances x 30 connections = 360 potential connections.
Autoscale to 24 instances -> 720.
The database does not automatically gain capacity.
```

WHY THIS WORKS

Pool size is not the same as useful database parallelism. Measure acquisition wait, active usage, transaction duration and DB saturation. Use short transactions, bounded acquisition and backpressure. Different workloads may require separate budgets even when they share one physical database.

COMMON PITFALL Raising pool size to fix waiting can worsen a CPU- or lock-bound database.

INTERVIEWER FOLLOW-UP

Which limit stops autoscaling from overwhelming the database?

Answer: Cap total connections across replicas: instances x maxPoolSize <= DB budget, with headroom for failover and admin work.

Why is ConcurrentHashMap not a distributed transaction?

SHORT INTERVIEW ANSWER

It provides thread-safe map operations within one JVM. It neither shares memory across instances nor atomically commits independent business resources with crash durability.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
map.putIfAbsent(requestKey, result); // one JVM
check balance; debit account; add audit record;
// these three operations need a business transaction
```

WHY THIS WORKS

Per-key compute operations can coordinate a narrow local update, but a sequence across keys is not automatically atomic. Process restart loses ordinary in-memory state. Use the map for caches or local coordination only when the design tolerates these boundaries.

COMMON PITFALL Thread-safe containers do not make a multi-step business operation thread-safe by themselves.

INTERVIEWER FOLLOW-UP

What changes when a second application instance handles the retry?

Answer: Another JVM has a different map and lock. Use durable shared state and database uniqueness for global correctness.

How do JVM memory and GC affect system design?

SHORT INTERVIEW ANSWER

Allocation rate, live data and pauses influence tail latency and capacity. Budget heap plus native memory, thread stacks and buffers, and use evidence before tuning collector settings.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Large request body -> allocation spike.
Unbounded queue -> retained tasks and payloads.
Container memory = heap + non-heap + native overhead.
```

WHY THIS WORKS

Use GC logs, allocation profiling, heap analysis and request traces to distinguish causes. Streaming payloads and bounding queues can solve a design problem that a larger heap only delays. Cache entries often contain object overhead beyond their serialized byte size.

COMMON PITFALL Setting maximum heap equal to the container memory limit leaves no room for other memory.

INTERVIEWER FOLLOW-UP

Which retained objects grow with backlog rather than useful working data?

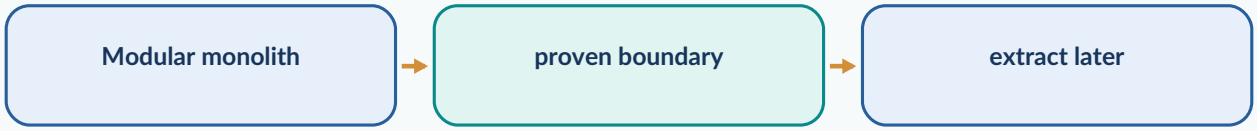
Answer: Queued events, retained payloads and unbounded collections may grow with backlog. Measure heap, allocation and GC pauses.

When should you choose a modular monolith over microservices?

SHORT INTERVIEW ANSWER

A modular monolith fits when one deployment and local transactions meet current needs. Microservices can help independent ownership or scaling but introduce network and operational costs.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Modules: catalog, orders, payment workflow.
Enforce interfaces and data ownership inside one application.
Extract notifications when independent scale justifies it.

WHY THIS WORKS

A monolith can be well structured; microservices can still be tightly coupled. Judge team boundaries, release needs, workloads and reliability. Avoid splitting every entity into a service, which can make one business operation require many coordinated calls.

COMMON PITFALL More network boundaries create more partial-failure states.

INTERVIEWER FOLLOW-UP

Which measurable problem will this service extraction solve?

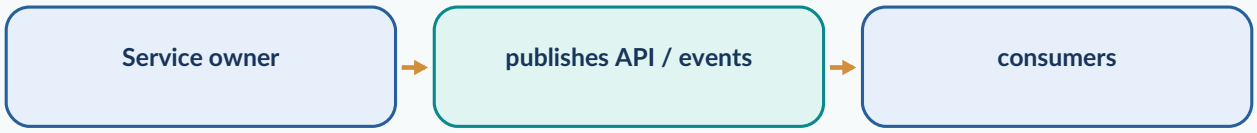
Answer: Name a measured need such as independent deployment, scaling or team ownership; extraction has operational cost.

How should a service own its data?

SHORT INTERVIEW ANSWER

One service should control the writes and invariant for its business domain. Other services use contracts or replicated projections rather than bypassing ownership with shared-table writes.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Inventory owns stock and reservations.
Order service calls reserve or consumes reservation outcomes.
Analytics reads a projection, not live ownership tables.

WHY THIS WORKS

Physical databases may initially be shared for operational reasons, but access boundaries should remain explicit. Cross-service foreign keys and direct updates make independent evolution harder. Define how data is copied and how deletion or correction reaches projections.

COMMON PITFALL Sharing a database does not justify every service modifying every table.

INTERVIEWER FOLLOW-UP

Who approves a schema change that affects another service's projection?

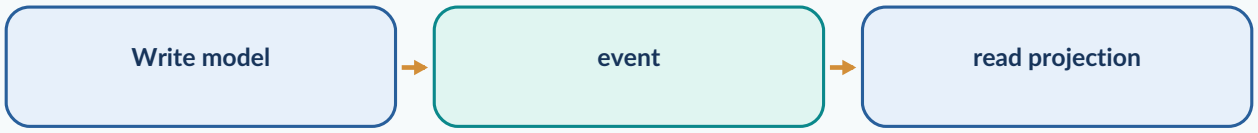
Answer: The owning service approves its schema. Consumers coordinate event/API contracts and rebuild their own projections.

What is CQRS, and when is it useful?

SHORT INTERVIEW ANSWER

CQRS separates command behavior from query behavior. It can use one database or separate projections; it is useful when read and write models have meaningfully different needs.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Command: reserve inventory with strict validation.
Query: product search with precomputed availability text.
Projection lag is shown or accepted by product policy.

WHY THIS WORKS

CQRS does not require event sourcing, Kafka or separate microservices. Separate read storage adds update propagation, rebuild and lag management. Start with simple command/query separation and add projections for a demonstrated read problem.

COMMON PITFALL A read projection should not silently become the authority for stock confirmation.

INTERVIEWER FOLLOW-UP

How will a newly deployed projection catch up?

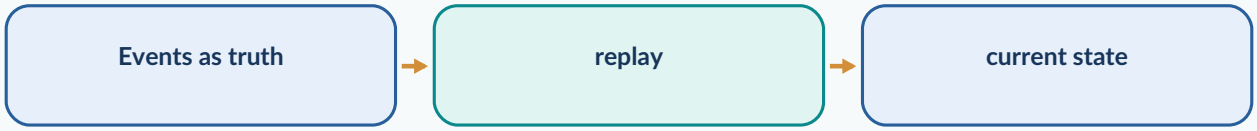
Answer: Replay a retained event log or backfill from a snapshot, then apply live changes until a measured lag threshold is met.

What is event sourcing, and how does it differ from an event log?

SHORT INTERVIEW ANSWER

Event sourcing stores domain events as the authoritative history from which current state is derived. A transport event log may carry copies of changes while another store remains authoritative.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Account events: Opened, Credited, Debited.
Current balance = replay of valid ordered account events.
Snapshot: acceleration, not replacement of authoritative history.

WHY THIS WORKS

The approach requires versioned events, aggregate concurrency, replay rules and handling of corrections. External effects must not automatically repeat during replay. Historical data deletion and sensitive fields need deliberate design. Event sourcing is useful for some domains but adds complexity.

COMMON PITFALL Publishing OrderCreated after a normal database insert is not automatically event sourcing.

INTERVIEWER FOLLOW-UP

How would you evolve an old event whose schema no longer matches current code?

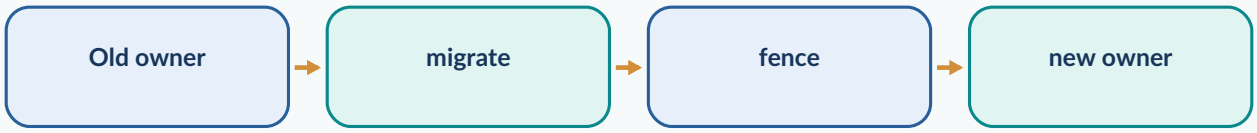
Answer: Use versioned event upcasters or adapters, preserve old event meaning and test replay against historical samples.

How can a large system migrate safely to a new design?

SHORT INTERVIEW ANSWER

Move one bounded capability at a time, define a single authority during transition, and verify equivalence before switching traffic. Use reversible routing and a repair path.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Step 1: new read model receives change events.
Step 2: shadow compare selected responses.
Step 3: canary traffic, observe, then expand.

WHY THIS WORKS

Dual writes across independent stores are fragile without a durable coordination mechanism. Migration checkpoints, validation counts and reconciliation reveal missed changes. After cutover, plan how rollback interacts with writes that occurred only in the new authority.

COMMON PITFALL Rolling back application code does not necessarily roll back changed data semantics.

INTERVIEWER FOLLOW-UP

What prevents both old and new services from owning the same write?

Answer: Use one write owner at a time; route commands through it and compare dual-read results before a staged cutover.

Why use a search index instead of the transactional database for text search?

SHORT INTERVIEW ANSWER

A search engine offers specialized text indexing, relevance and query features. Keep authoritative data ownership separate and define how updates reach the searchable projection.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Product DB commit -> durable change event -> index update.
Search returns product IDs and display fields.
Checkout revalidates price and stock from authoritative services.
```

WHY THIS WORKS

The index may lag or contain a product that was just deleted. Handle tombstones, versions and authorization filters. For simpler workloads, the database's built-in text features may be sufficient; a separate search cluster creates operational overhead.

COMMON PITFALL A relevant search hit is not proof that the user may access or purchase the item.

INTERVIEWER FOLLOW-UP

How will you rebuild the index without blocking live search?

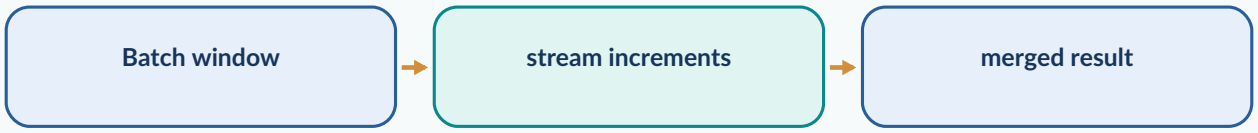
Answer: Build a new index from a snapshot while streaming live updates, compare it, then atomically switch the search alias.

How do batch and stream processing differ?

SHORT INTERVIEW ANSWER

Batch processes bounded datasets at intervals. Stream processing handles ongoing events with incremental state. Choose based on freshness requirements, cost and correctness under late data.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Daily revenue report: batch may be enough.
Fraud signal during checkout: timely stream may be needed.
Both need rules for corrections and duplicated events.

WHY THIS WORKS

A hybrid design can use streaming for rapid visibility and batch reconciliation for authoritative correction. Streaming does not inherently mean real-time completion; backlog and downstream delays still matter. Track data freshness independently of request latency.

COMMON PITFALL Fast ingestion does not establish accurate or timely final aggregates.

INTERVIEWER FOLLOW-UP

How late can an event arrive and still revise the result?

Answer: Choose a lateness watermark and correction policy. Very late events may update historical results or go to adjustments.

What is the difference between event time and processing time?

SHORT INTERVIEW ANSWER

Event time describes when the business event occurred. Processing time describes when a processor observed it. Delays and clock issues can make those times differ substantially.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Purchase happened at 23:59 yesterday.
Mobile device reconnects at 08:00 today.
A daily sales metric must choose the intended time basis.

WHY THIS WORKS

Define timezone, trustworthy timestamp source and late-event handling. Watermarks or lateness policies guide when a computation treats a window as sufficiently complete; they cannot make all late events impossible. Reprocessing may update earlier totals.

COMMON PITFALL Device clocks are not necessarily accurate enough to enforce business ordering.

INTERVIEWER FOLLOW-UP

Should a late refund alter yesterday's report or today's adjustment ledger?

Answer: For finance, keep an immutable original period and post a dated correction if accounting rules require that approach.

How can you maintain a materialized aggregate safely?

SHORT INTERVIEW ANSWER

Define the source of truth, stable event identities, update ordering and recovery strategy. Apply each local increment with deduplication or rebuild from authoritative records.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Event e-5: order total + 1,250 minor units.
Atomic transaction: insert processed event ID, update total.
Duplicate e-5: no additional increment.

WHY THIS WORKS

This is a local database pattern, not universal exactly-once streaming. Corrections need explicit negative or replacement operations. High-volume counters may be partitioned and merged when delayed totals are acceptable. Decide what the dashboard should display during lag or rebuild.

COMMON PITFALL Summing repeated delivery records without deduplication overstates revenue.

INTERVIEWER FOLLOW-UP

How will you reconcile the aggregate against the ledger?

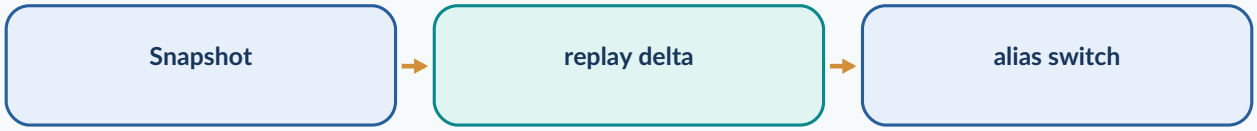
Answer: Replay ordered IDs from the ledger into a versioned aggregate and compare sums/record counts to authoritative rows.

How should you rebuild a projection while writes continue?

SHORT INTERVIEW ANSWER

Take a consistent baseline or a defined source position, replay changes from the matching position, and switch only after checking catch-up and correctness. Track versions to avoid older replay overwriting newer data.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Build index-v2 from a baseline at source position P.
Replay changes after P while live writes continue.
Validate and switch a read alias when caught up.

WHY THIS WORKS

Snapshot and change positions must align; otherwise the gap can lose or duplicate changes. Deduplication handles duplicates, while version checks preserve latest state. Tombstones matter as much as inserts. Use resource budgets so rebuilding does not starve production traffic.

COMMON PITFALL Copying rows and then listening for events without a gap strategy can miss writes.

INTERVIEWER FOLLOW-UP

What evidence shows the new projection is complete enough for cutover?

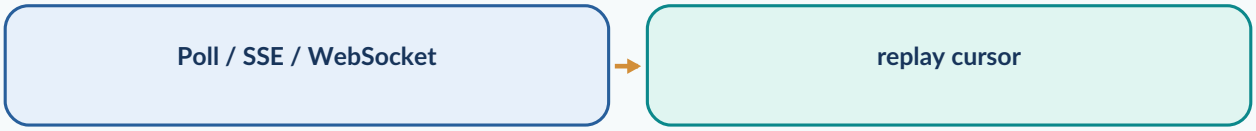
Answer: Verify source coverage, event catch-up position, counts and sampled results; then switch reads and monitor mismatches.

When should you use polling, server-sent events or WebSockets?

SHORT INTERVIEW ANSWER

Polling is simple for infrequent updates; server-sent events provide server-to-client streaming; WebSockets support bidirectional communication. Choose according to interaction, infrastructure and recovery needs.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Status every minute: polling.
Live job progress: SSE may fit.
Interactive chat: WebSocket plus durable history API.

WHY THIS WORKS

Persistent connections require authentication renewal, heartbeat policy, load balancing and draining. Reconnect should resume from a stable position rather than assume nothing happened while disconnected. The transport does not store messages or prove delivery to a person.

COMMON PITFALL A successful socket write does not prove that a user read a message.

INTERVIEWER FOLLOW-UP

How does the client recover messages missed during reconnect?

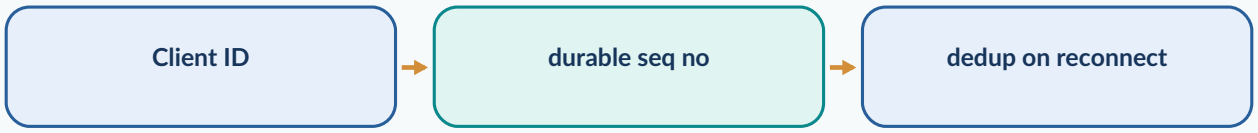
Answer: Keep a last-seen event ID/sequence. On reconnect, fetch the missing interval and deduplicate delivered events.

How should a chat system define message ordering?

SHORT INTERVIEW ANSWER

Use an explicit conversation sequence or another deterministic contract. Distinguish server acceptance order, delivery order and display order, and do not rely solely on client clocks.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Client sends clientMessageId = m-7.
Server commits conversation sequence = 104.
Retry m-7 returns the same accepted message.

WHY THIS WORKS

Serializing sequence assignment per conversation can become a hot partition for a huge room. If strict global ordering is unnecessary, relax the requirement explicitly. Delivery through multiple gateways may differ, so clients can sort or detect gaps using the server sequence.

COMMON PITFALL Two timestamps can be equal or inaccurate and still need a tie-breaker.

INTERVIEWER FOLLOW-UP

What happens when sequence 106 arrives before 105?

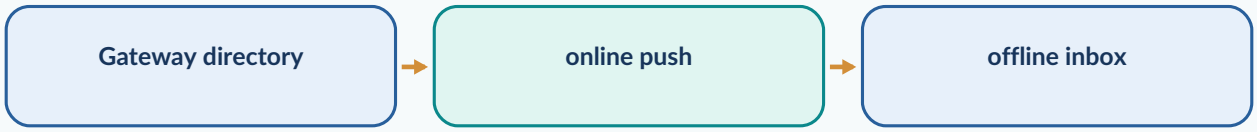
Answer: Buffer version 106 briefly until 105 arrives, then replay missing history; do not assume network arrival order is truth.

How do you route messages to connected users?

SHORT INTERVIEW ANSWER

Track which connection gateway currently hosts a user's sessions, route live delivery there, and keep durable message history independent of the connection directory.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Directory: user-42 -> gateway-A + gateway-C.
Message committed -> delivery event -> active gateways.
Offline or missed delivery -> recover from history cursor.
```

WHY THIS WORKS

Directory entries expire and can become stale during disconnection or deployment. Multi-device users may need delivery to several sessions. A failed live route should not delete the durable message. Presence is often approximate and should have a TTL-based contract.

COMMON PITFALL Presence is a transient observation, not proof that a person is attentive.

INTERVIEWER FOLLOW-UP

How will a stale gateway directory entry affect message recovery?

Answer: The first push may fail, but durable messages remain in storage. Reconnect and replay must not rely on that directory.

How do fan-out on write and fan-out on read compare?

SHORT INTERVIEW ANSWER

Fan-out on write precomputes recipient feeds, making reads cheaper. Fan-out on read combines sources at read time, making writes cheaper but reads more expensive. A hybrid handles uneven audience sizes.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Normal author: add post ID to follower feed projections.
Celebrity author: merge recent posts during feed read.
Rank and paginate using a stable feed contract.

WHY THIS WORKS

Audience size and read frequency decide the tradeoff. Precomputed fan-out needs retry, duplicate handling and deletion propagation. Read-time merging needs bounded source queries and caching. Ranking changes can complicate cursor semantics.

COMMON PITFALL A post with millions of followers can overwhelm naive synchronous fan-out.

INTERVIEWER FOLLOW-UP

How long may a follower wait before a new post appears?

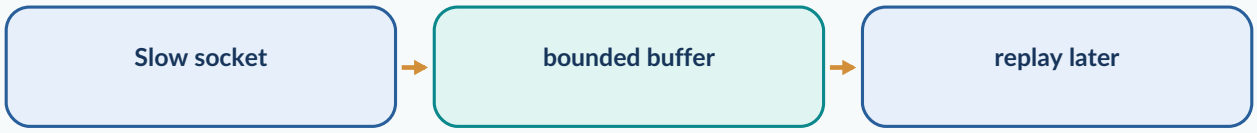
Answer: State an explicit freshness SLO, e.g., a few seconds for normal feed updates, and test it under fan-out load.

How do you control slow realtime clients?

SHORT INTERVIEW ANSWER

Bound per-connection buffers and define disconnect, resume or coalescing behavior when a client cannot keep up. Preserve durable events separately from ephemeral updates.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Chat messages: durable history with resume cursor.
Typing indicators: coalesce or drop if the buffer is full.
Slow client: disconnect with a recoverable session policy.

WHY THIS WORKS

Do not let one connection retain unlimited heap or block a shared event loop. Backpressure policies vary by message type; a missed typing indicator is less serious than a missing committed message. Monitor queue bytes and age, not just message count.

COMMON PITFALL An unbounded per-socket queue can exhaust a server before CPU is busy.

INTERVIEWER FOLLOW-UP

Which events are safe to drop and which require replay?

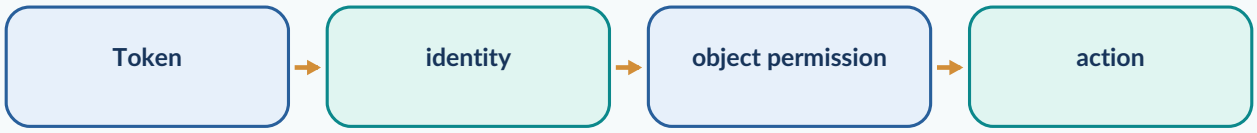
Answer: Drop typing indicators or presence updates if allowed; retain chat messages and replay by durable ID.

How do authentication and authorization differ?

SHORT INTERVIEW ANSWER

Authentication establishes identity; authorization decides whether that identity may perform this action on this resource. Validate both for synchronous and asynchronous work.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
GET /orders/42
Authenticated user: customer-7.
Service checks that order 42 belongs to customer-7 or an allowed role.
```

WHY THIS WORKS

A gateway can validate a token, but the owning service still knows the object-level business rule. Scope background jobs to the original authority or an explicit service identity. Permission changes may require fresh checks rather than trusting old cached decisions.

COMMON PITFALL Knowing an unguessable resource ID does not grant permission to access it.

INTERVIEWER FOLLOW-UP

What happens if a queued command runs after permission is revoked?

Answer: Recheck permission at execution, or cancel the command if authorization was revoked; queue-time checks may become stale.

How should a multitenant system isolate data?

SHORT INTERVIEW ANSWER

Bind every operation to a validated tenant context and enforce isolation in queries, keys, caches, events and operational tooling. Larger or sensitive tenants may need stronger physical separation.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
SELECT * FROM orders
WHERE tenant_id = :authorizedTenant AND id = :orderId;
Cache key: tenantId + ':' + orderId.
```

WHY THIS WORKS

Do not trust a tenant ID from the request without connecting it to the caller's authority. Tenant quotas and workload budgets protect performance isolation as well as privacy. Consider row-level controls, separate schemas or dedicated databases according to requirements and operating cost.

COMMON PITFALL A correctly scoped SQL query can still leak data through an incorrectly scoped cache.

INTERVIEWER FOLLOW-UP

How are support tools prevented from bypassing tenant boundaries?

Answer: Use tenant-aware repository policies, row-level security where suitable, scoped support roles and audited access.

Why are encryption, secrets and key rotation separate design concerns?

SHORT INTERVIEW ANSWER

Transport and storage encryption protect different paths. Secret management controls credentials, and key rotation limits long-lived exposure. Each requires an ownership and rollout plan.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

TLS: protects request bytes in transit.
DB/object encryption: protects stored bytes under its threat model.
Service credential: delivered securely and rotated without downtime.

WHY THIS WORKS

Encryption does not prevent an authorized application from returning the wrong user's data. Avoid placing secrets in source code, logs or ordinary config artifacts. Design overlapping credential validity and revocation so services can transition safely.

COMMON PITFALL A token in a trace or error log can bypass otherwise strong transport protection.

INTERVIEWER FOLLOW-UP

How will old and new keys coexist during rotation?

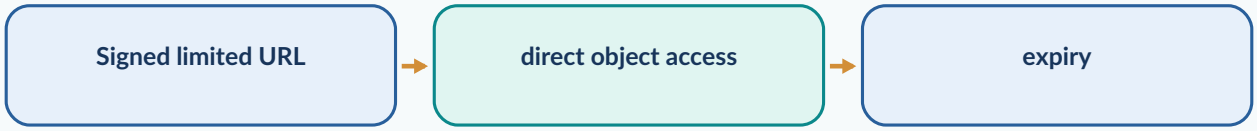
Answer: Publish a new key ID, accept old and new during migration, re-encrypt/expire old data, then revoke the old key.

How should signed upload or download URLs be secured?

SHORT INTERVIEW ANSWER

Authorize the request before issuing a narrowly scoped, short-lived capability. Bind the operation to a generated object key and validate content and ownership again during finalization.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

User requests upload for an owned draft asset.
Server issues URL for that specific object and operation.
Finalization checks object metadata and approved workflow state.

WHY THIS WORKS

Treat the URL as a bearer capability and avoid logging it. Depending on the storage API, restrictions and reuse behavior differ; expiry is not always a one-use guarantee. Size, type, checksum and scanning requirements should be enforced by supported policy or validation.

COMMON PITFALL A client-provided object key can overwrite another user's content if accepted blindly.

INTERVIEWER FOLLOW-UP

Can an issued URL be reused before it expires?

Answer: Often yes until expiry, depending on the signing policy. Use short TTLs, narrow scope and storage-level controls.

How should deletion work when data has replicas and projections?

SHORT INTERVIEW ANSWER

Define a deletion lifecycle covering authoritative rows, search indexes, caches, objects, events, exports and backups. Track completion and prevent deleted data from reappearing during replay.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Mark account deletion request durably.
Propagate tombstones to projections and storage owners.
Rebuild logic respects tombstones and retention policy.

WHY THIS WORKS

Some audit or financial records may follow separate documented retention requirements; this guide does not prescribe legal policy. Minimize sensitive fields in immutable events before creating them. Backup expiry and restore procedures must respect the intended deletion lifecycle.

COMMON PITFALL Removing one database row does not delete copies in a search cluster or cache.

INTERVIEWER FOLLOW-UP

What prevents a replay from recreating a deleted user profile?

Answer: Record tombstones/deletion versions and prevent event replay from recreating them; purge dependent projections.

How do metrics, logs and traces complement each other?

SHORT INTERVIEW ANSWER

Metrics summarize behavior over time; logs record detailed events; traces connect stages of a distributed operation. Use stable correlation IDs and consistent semantic fields.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Metric: checkout latency histogram.
Log: order=o-7, state=PAYMENT_PENDING, reason=timeout.
Trace: gateway -> order -> inventory -> provider.
```

WHY THIS WORKS

Not every request needs a verbose log or stored trace. Sampling, cardinality and retention affect cost and diagnostic coverage. Carry trace context across messages and record the relationship between acceptance and later processing. Avoid logging payloads with secrets.

COMMON PITFALL A trace that starts after a long queue wait can understate the user latency.

INTERVIEWER FOLLOW-UP

Which signal links an HTTP request to its asynchronous workflow?

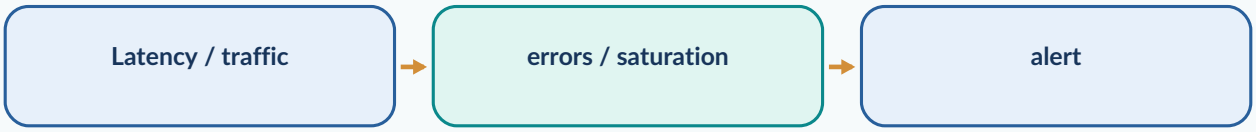
Answer: Use one correlation/trace ID plus the durable operation/event ID across HTTP, outbox and consumer logs.

Which golden signals should you monitor?

SHORT INTERVIEW ANSWER

Track latency, traffic, errors and saturation, plus domain-specific correctness and freshness. Pair user symptoms with resource causes so alerts support diagnosis.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

User signals: successful bookings and confirmation latency.
Resource signals: DB lock wait, pool wait and queue age.
Correctness: oversell count and unresolved payment outcomes.

WHY THIS WORKS

CPU alone can look healthy while requests wait on a database or remote service. A business invariant alert may matter more than a small latency increase. Define ownership and runbooks for actionable alerts and suppress noisy non-actionable events.

COMMON PITFALL A broker being reachable does not mean consumers are keeping up.

INTERVIEWER FOLLOW-UP

Which alert fires when accepted work stops progressing?

Answer: Alert on oldest pending age, processing success rate and stuck-state counts, not only API success at acceptance.

Why can metric cardinality become a production problem?

SHORT INTERVIEW ANSWER

Every unique label combination can create another time series. Unbounded labels such as user ID or request ID increase memory, storage and query costs.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Good bounded label: operation='checkout', result='timeout'.
Risky label: request_id='unique-for-every-request'.
Put detailed identifiers in controlled logs or traces instead.

WHY THIS WORKS

Even several individually bounded labels can multiply into a large set. Establish budgets for tenant, route and exception labels. Normalize dynamic URL paths into route templates. Use exemplars or trace links when supported to connect a metric spike to diagnostic detail.

COMMON PITFALL Labeling by raw URL can create one series for every object ID.

INTERVIEWER FOLLOW-UP

How many combinations can this label schema produce?

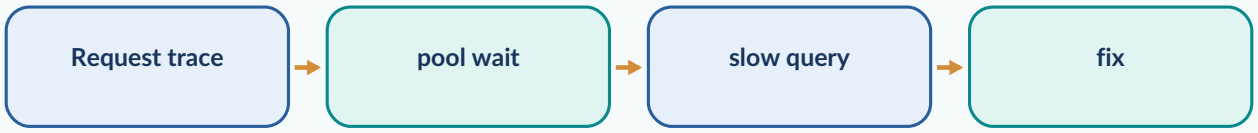
Answer: Multiply the number of unique values for all label dimensions; avoid unbounded user_id/ request_id metric labels.

How do you find the bottleneck in a slow Java API?

SHORT INTERVIEW ANSWER

Break down queue, executor, connection acquisition, database, remote call and serialization time. Correlate traces with resource metrics and profiles before changing pool or heap settings.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Observed p99 = 1.8 s.
Trace: pool acquisition = 1.5 s; SQL = 20 ms.
Investigate long transactions and connection budget first.
```

WHY THIS WORKS

If SQL itself is slow, inspect plans, row counts and lock waits. If CPU is busy, profile allocation and compute paths. Test a candidate change under representative load and compare the user SLO, not only one local metric.

COMMON PITFALL Increasing threads without locating the wait can increase downstream contention.

INTERVIEWER FOLLOW-UP

Is the connection pool empty because of slow queries, leaks or long transactions?

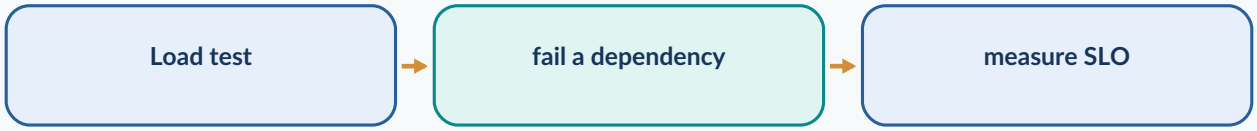
Answer: Check active/idle/awaiting connections, slow-query times, blocked transactions and leaks with trace evidence.

How should you validate capacity and failure behavior?

SHORT INTERVIEW ANSWER

Use representative data and workload distributions, measure sustainable throughput and tails, then inject specific failures. Verify invariants and recovery, not just happy-path request counts.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Load: normal + burst + hot tenant + large payload.
Fault: kill consumer after DB commit before acknowledgement.
Expected: duplicate delivery but one local business effect.

WHY THIS WORKS

Include cold cache, dependency slowness and recovery backlog. A small deterministic lab proves an algorithmic behavior, not production capacity or distributed correctness. Document environment and assumptions so measurements are reproducible.

COMMON PITFALL A benchmark with only cache hits cannot size the database fallback path.

INTERVIEWER FOLLOW-UP

Can recovery traffic coexist with live traffic without missing the SLO?

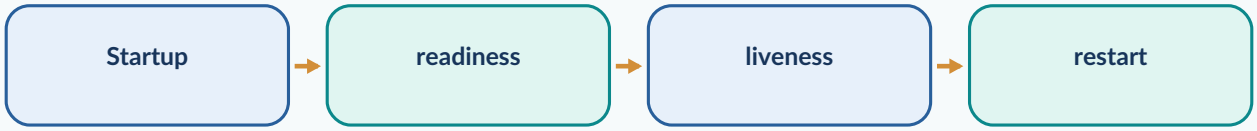
Answer: Run load plus failover/replay together, measuring user SLOs and recovery times, not only steady-state throughput.

How do liveness, readiness and startup probes differ?

SHORT INTERVIEW ANSWER

Liveness guides restart of a stuck process; readiness guides traffic admission; startup protects slow initialization before other probes begin. Use each for its intended lifecycle question.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Startup: initialization completed.
Readiness: able to handle this service's traffic.
Liveness: process is irrecoverably unhealthy without restart.

WHY THIS WORKS

Probe semantics and orchestration actions must match the platform. Making liveness depend on every remote service can cause cascading restarts during a shared outage. Readiness can reflect critical dependencies while optional features degrade. Avoid expensive probes that worsen overload.

COMMON PITFALL A temporarily unavailable database is not always a reason to restart every API process.

INTERVIEWER FOLLOW-UP

How should readiness behave during connection-pool exhaustion?

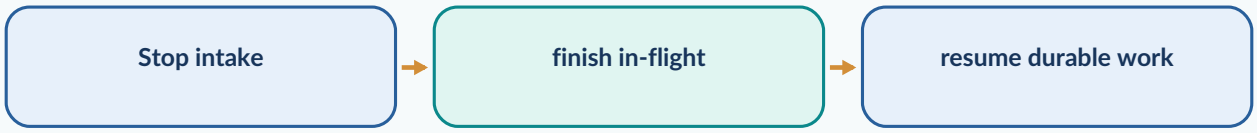
Answer: Mark unready when the node cannot serve essential requests; prevent tight readiness flapping with sane thresholds.

What does graceful shutdown require?

SHORT INTERVIEW ANSWER

Stop taking new work, remove the instance from traffic, drain or checkpoint existing work within a budget, and release resources. Durable uncompleted work should be safely redeliverable.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Mark unready -> propagate routing change -> stop admission.
Finish short requests; checkpoint/ack completed consumer work.
Close pools before the termination deadline.
```

WHY THIS WORKS

Load balancer propagation and existing connections introduce races. For long-lived sockets, communicate reconnect and provide resume positions. Never acknowledge unfinished message effects just to make shutdown faster. A forced kill remains possible, so correctness must survive it.

COMMON PITFALL Graceful shutdown is an optimization for clean exits, not a replacement for crash recovery.

INTERVIEWER FOLLOW-UP

What happens when termination arrives before a payment workflow finishes?

Answer: Persist workflow state first. On termination, stop admitting work, drain briefly, then resume from durable state later.

How do rolling, canary and blue-green deployment differ?

SHORT INTERVIEW ANSWER

Rolling replaces instances gradually; canary exposes a small portion of traffic first; blue-green keeps two deployment environments and switches routing. All need compatible data and contracts.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Canary: send 1% of checkout traffic to version B.
Compare error rate, latency and business invariants.
Increase traffic only after the defined observation gate.

WHY THIS WORKS

Percentages are illustrative, not a fixed safe policy. Mixed versions can share queues and schemas, so validate compatibility in both directions where required. Rollback must consider new data states and side effects, not only code images.

COMMON PITFALL A healthy process startup is not proof that a deployment is safe for users.

INTERVIEWER FOLLOW-UP

What signal detects a canary creating duplicate reservations?

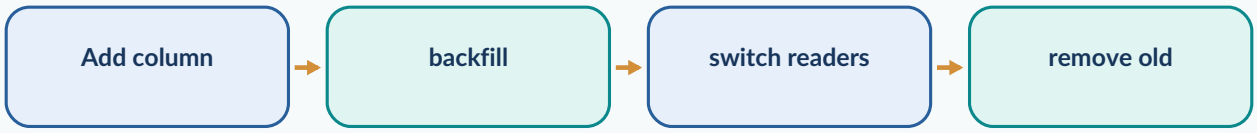
Answer: Compare canary duplicate-key conflicts, duplicate reservations per request ID and invariant alerts; abort if they rise.

What is an expand-and-contract database migration?

SHORT INTERVIEW ANSWER

Add compatible structures first, migrate readers and writers through a transition, validate backfill, then remove old structures only after old consumers are gone.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
1. Add nullable delivery_window_v2.
2. Deploy compatible writers; backfill with checkpoints.
3. Switch reads after validation.
4. Later enforce constraints and remove old field.
```

WHY THIS WORKS

DDL locking and online behavior differ by database and operation. Avoid a single unbounded backfill transaction. A dual-write phase within one local transaction can be manageable; across independent databases it needs additional coordination. Test old application code against the expanded schema.

COMMON PITFALL Dropping the old column before the last old instance stops can break a rolling deploy.

INTERVIEWER FOLLOW-UP

How will you resume a backfill after interruption?

Answer: Backfill in small, idempotent batches with a checkpoint/cursor and verification so restart resumes safely.

How do RPO and RTO differ, and why are replicas not enough?

SHORT INTERVIEW ANSWER

RPO is the acceptable data-loss window; RTO is the acceptable recovery time. Replicas can copy corruption or accidental deletion, so recovery also needs protected backups and tested restoration.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Hypothetical target: RPO 5 minutes; RTO 30 minutes.
Need compatible backup/log retention and restore process.
Test actual restore time and recovered data integrity.
```

WHY THIS WORKS

Targets are requirements, not guarantees produced by naming a replication mode. Include credentials, configuration, object data and dependencies in the recovery plan. A failover drill and a backup restore drill verify different failure scenarios.

COMMON PITFALL Three live copies can all reproduce the same erroneous delete.

INTERVIEWER FOLLOW-UP

How do you restore a correct point in time without losing later valid work unnecessarily?

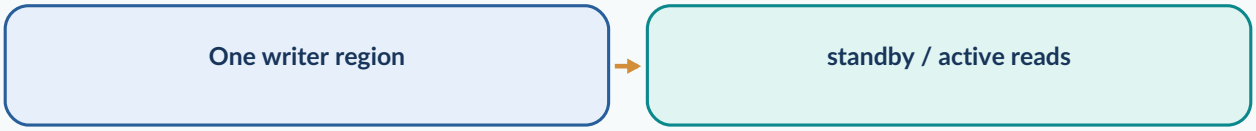
Answer: Restore from backup and WAL to a verified point; reconcile later valid operations from independent logs/ providers where possible.

How do active-passive and active-active regions differ?

SHORT INTERVIEW ANSWER

Active-passive routes normal writes to one active region and keeps another for recovery. Active-active serves work in multiple regions and needs explicit data ownership or conflict resolution.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Active-passive: simpler write authority, slower failover path.
Active-active: local service paths, harder concurrent-write rules.
Home-region model: each account has one write owner.

WHY THIS WORKS

The right design depends on recovery targets, latency and the business invariant. Active-active application traffic does not necessarily imply multi-primary writes to every record. Cross-region synchronous coordination can increase write latency and reduce progress during partition.

COMMON PITFALL Two regions accepting the same account debit independently can violate balance rules.

INTERVIEWER FOLLOW-UP

Where is the write authority for a user traveling abroad?

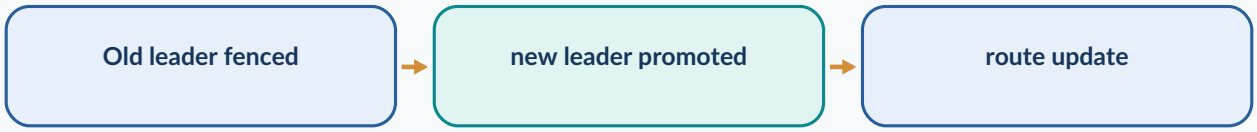
Answer: Route to an explicit home/write authority. Traveling users may read locally but send strict writes to their owner.

Why is failover more than changing DNS?

SHORT INTERVIEW ANSWER

Failover must ensure the new authority is ready, old writers cannot continue conflicting writes, data loss is understood, and clients and dependent systems reconnect safely.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Detect failure -> confirm authority decision.
Fence old writer -> promote suitable replica.
Update routing -> reconcile pending external operations.
```

WHY THIS WORKS

DNS caching and existing connections make traffic changes gradual. Promotion after asynchronous replication may lose acknowledged writes under some configurations. Test role changes, retries and idempotency during failover. A control-plane error can be more dangerous than a short outage.

COMMON PITFALL Promoting a replica without preventing old-primary writes can create split brain.

INTERVIEWER FOLLOW-UP

How are requests to the old region rejected after promotion?

Answer: Use leader epochs, fencing tokens or storage leases so a demoted region cannot accept writes after promotion.

What does data locality change in a design?

SHORT INTERVIEW ANSWER

Placing data near its main access path can reduce latency, transfer cost and coordination. It also affects partition routing, recovery and requirements for where particular data may reside.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Tenant home region stores authoritative records.
Global directory maps tenant -> home region.
Public static assets can be replicated near readers.
```

WHY THIS WORKS

Directory updates become part of tenant migration. Region requirements may arise from contracts or policy; verify the applicable requirement rather than assuming all data must be globally copied. Model cross-region access explicitly for users and background workers.

COMMON PITFALL A global cache can undermine an otherwise carefully scoped locality policy.

INTERVIEWER FOLLOW-UP

How do you move a tenant without two simultaneous write owners?

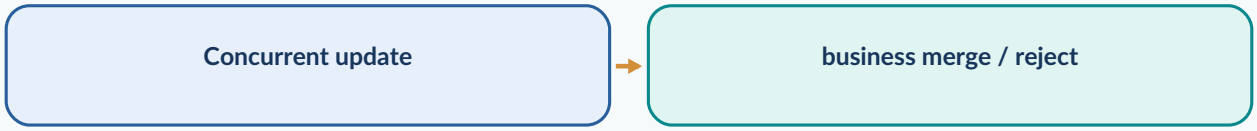
Answer: Pause its writes, copy/verify state, flip versioned routing, then enable one new owner; reject stale route versions.

How can conflict resolution change business meaning?

SHORT INTERVIEW ANSWER

Last-write-wins, merging and conflict-preserving strategies each encode a policy. Choose a strategy that matches the data, rather than using timestamps as a universal correctness solution.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Profile nickname: last-write-wins may be acceptable.
Set of likes: merge identities under a defined model.
Account balance: arbitrary last-write-wins loses valid money movements.

WHY THIS WORKS

Wall-clock skew and ties can distort timestamp ordering. Some data types support useful convergent merge semantics, but reversals and business constraints still require care. For critical operations, avoid conflicting independent writers or use an appropriate coordinated protocol.

COMMON PITFALL Eventual convergence does not establish that the converged value is correct for the business.

INTERVIEWER FOLLOW-UP

Can two concurrent valid operations both be preserved?

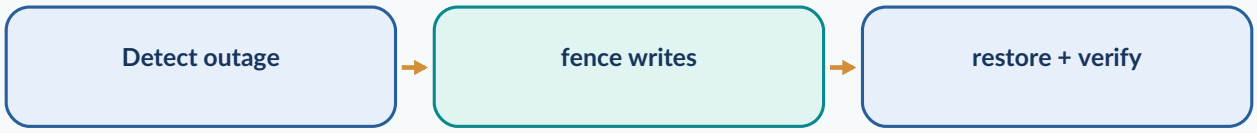
Answer: Some operations are commutative (two increments); two purchases for one remaining item are not. Merge only with business rules.

What should a region-outage runbook include?

SHORT INTERVIEW ANSWER

Describe detection, authority and fencing, promotion, traffic admission, dependency readiness, recovery validation and eventual return. Assign owners and define a stopping condition for unsafe ambiguity.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Check replication position and expected loss window.
Verify identity, secrets, DB, broker and object access.
Track pending workflows through stable request IDs.

WHY THIS WORKS

Practice with partial outages, not just complete region disappearance. Returning to the old topology needs synchronization and ownership transfer just like initial failover. Communication and status systems should not rely only on the failing region.

COMMON PITFALL A second region without tested credentials and dependencies is not a complete recovery capability.

INTERVIEWER FOLLOW-UP

How will you prove the system is safe before restoring normal write traffic?

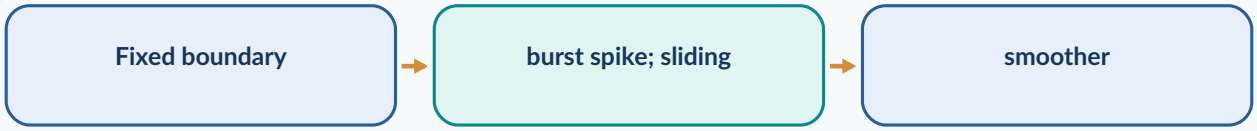
Answer: Identify owners, promotion steps, write fencing, data checks, DNS/client routing, rollback, contacts and drills.

How do fixed-window and sliding-window rate limits compare?

SHORT INTERVIEW ANSWER

Fixed windows count requests in discrete periods and are simple, but allow boundary bursts. Sliding-window approaches smooth the boundary with more state or an approximation.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Fixed limit: 100/minute.
100 requests at 12:00:59 and 100 at 12:01:00:
200 requests can arrive within two seconds.

WHY THIS WORKS

A sliding log stores request times for exact rolling counts but uses memory proportional to traffic. Bucketed or weighted counters reduce cost with approximate behavior. Choose precision according to abuse risk and budget. The rejection response should explain retry behavior when useful.

COMMON PITFALL A per-minute quota does not necessarily protect short-term concurrency.

INTERVIEWER FOLLOW-UP

What burst can a caller create near a window boundary?

Answer: A caller can send nearly a full limit just before reset and another full limit just after it: almost 2x briefly.

How does a token bucket work?

SHORT INTERVIEW ANSWER

Tokens accumulate up to a capacity; each admitted request consumes tokens. The refill rate controls long-term throughput and capacity controls the permitted burst.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Capacity = 10 tokens; refill = 2 tokens/s.
Initially allow a burst of 10.
After exhaustion, roughly 2 new unit requests/s can proceed.
```

WHY THIS WORKS

Use an appropriate monotonic time source for elapsed time in a local limiter. Concurrent checks and consumption must be atomic. Weighted operations can consume different token amounts. A distributed implementation needs a shared atomic state transition or an explicitly approximate policy.

COMMON PITFALL Running this bucket independently in five instances multiplies the aggregate permitted rate.

INTERVIEWER FOLLOW-UP

What should happen when the request costs more than bucket capacity?

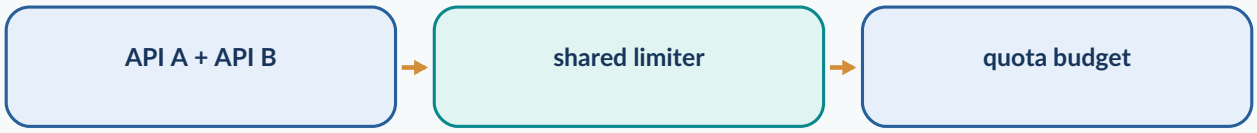
Answer: Reject an oversized request rather than wait forever; validate cost > 0 and document whether partial tokens are allowed.

How do you enforce a distributed quota?

SHORT INTERVIEW ANSWER

Choose a single authoritative state transition per quota key, or explicitly allocate approximate budgets to nodes. Define accuracy, latency and behavior when the coordinator is unavailable.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Key: tenant + operation + policyVersion.
Atomic check/refill/decrement in shared storage.
Local emergency concurrency caps protect each process.

WHY THIS WORKS

An atomic Redis script or dedicated quota service can coordinate a key, but storage failover and partition behavior still matter. For some APIs, temporary approximate quotas are acceptable; for financial limits, they may not be. Global coordination can become a latency or hot-key bottleneck.

COMMON PITFALL An eventually consistent read-then-write counter does not provide an exact global quota.

INTERVIEWER FOLLOW-UP

Which policy is used during a quota-store outage?

Answer: Use a central atomic counter/script or partitioned quota allocation; define consistency and fallback when unavailable.

How do quotas and fairness differ?

SHORT INTERVIEW ANSWER

A quota caps usage; fairness prevents one workload from starving others within shared capacity. Use per-tenant budgets, bounded queues or scheduling policies according to the product contract.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Tenant A: 10,000 queued reports.
Tenant B: 1 interactive request.
A single FIFO can make B wait behind all of A's work.

WHY THIS WORKS

Separate interactive and batch traffic or use weighted scheduling. Reserve capacity for control and recovery work when necessary. Tenant-specific limits should avoid high-cardinality metric explosions and have a manageable configuration lifecycle.

COMMON PITFALL Global rate limiting can protect the system while still allowing one tenant to dominate it.

INTERVIEWER FOLLOW-UP

What maximum wait can a small tenant experience during a large import?

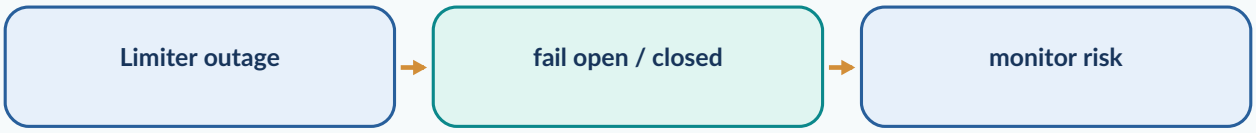
Answer: Use per-tenant queues/weights and maximum waits; a big import must not monopolize shared capacity.

Should a limiter fail open or fail closed?

SHORT INTERVIEW ANSWER

Choose per operation based on the consequence of excess work versus denied work. Document the fallback, apply local protection, and expose degraded enforcement to monitoring.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Public product reads: may use a cautious local fallback.
Paid external SMS: tighter admission may be required.
Critical money limit: do not invent availability guarantees.

WHY THIS WORKS

There is no universal safe setting. A fail-open policy can overload an already damaged dependency; a fail-closed policy can create an outage from the limiter itself. Use cached policy and bounded local budgets when appropriate, with clear accuracy limits.

COMMON PITFALL Treating every quota as a security invariant can unnecessarily reject harmless reads.

INTERVIEWER FOLLOW-UP

How long may degraded quota enforcement continue?

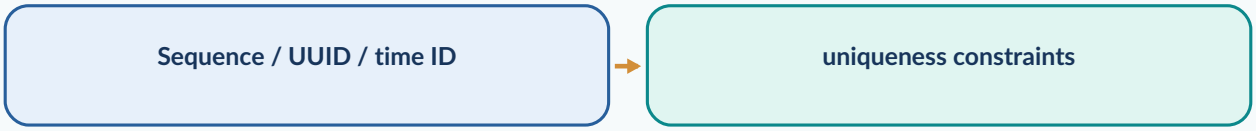
Answer: Only a short, documented period with extra alarms and reduced access, based on risk and capacity.

How do database sequences, random IDs and time-based IDs compare?

SHORT INTERVIEW ANSWER

Sequences simplify compact uniqueness in one authority; random IDs enable decentralized generation; time-based layouts can improve ordering but need clock and node-identity handling.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Sequence -> compact, may reveal volume.
Random UUID -> decentralized, larger keys.
Time-based ID -> timestamp + node + local sequence.
```

WHY THIS WORKS

Uniqueness and unpredictability are separate requirements. A unique ID is not an access-control mechanism. Distributed time-based generators need defined behavior on clock rollback, sequence exhaustion and duplicated node identifiers. Index locality and storage overhead should influence the choice.

COMMON PITFALL Encoding a sequential number does not make it secret.

INTERVIEWER FOLLOW-UP

What happens if two nodes accidentally use the same generator identity?

Answer: Detect duplicate generator IDs, fence the node, and use collision-resistant IDs or a durable unique constraint.

Why use Base62 for short codes, and what does it not solve?

SHORT INTERVIEW ANSWER

Base62 represents numbers with letters and digits to make a compact URL-friendly code. Encoding alone neither creates unique IDs nor protects against guessing.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Alphabet size = 62.
Six characters offer 62^6 = 56,800,235,584 combinations.
Random selection still needs collision detection and retry.

WHY THIS WORKS

Use a unique database constraint on the code and an appropriate allocation strategy. Sequential IDs encoded in Base62 avoid random collisions but can reveal patterns. Add moderation and redirect safety rules independently. Confirm allowed characters and case sensitivity across clients and storage.

COMMON PITFALL A huge namespace does not make random collision probability zero.

INTERVIEWER FOLLOW-UP

How would you allocate a user-selected custom alias safely?

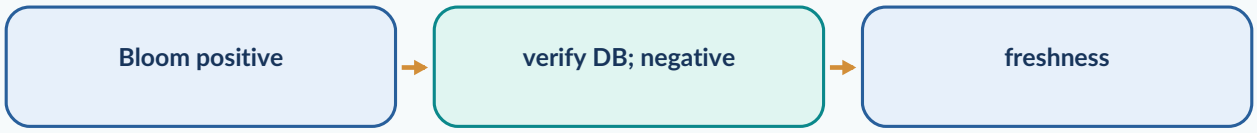
Answer: Enforce a unique alias index/conditional insert and return conflict when someone else owns it.

What is a Bloom filter useful for?

SHORT INTERVIEW ANSWER

It is a compact probabilistic membership structure that can say definitely absent or possibly present, under its normal no-deletion and consistent-population assumptions. Verify positive results in authoritative storage.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Request for code XYZ.
Filter says absent -> skip lookup only if filter coverage is valid.
Filter says present -> check the database.
```

WHY THIS WORKS

False positives create extra lookups. Stale population, unsafe deletion or incomplete rebuilds can create application-level false negatives even when the core algorithm does not. Estimate capacity and false-positive rate before choosing bit and hash counts.

COMMON PITFALL Never use a possibly-present answer as proof that a record exists.

INTERVIEWER FOLLOW-UP

What happens to newly created records before the filter update arrives?

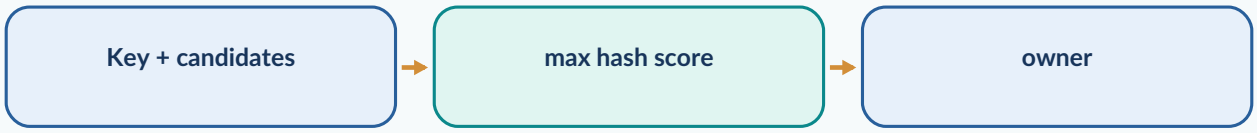
Answer: A stale filter can say 'not found' for a new record. Read source-of-truth during refresh or choose a freshness protocol.

When is rendezvous hashing useful?

SHORT INTERVIEW ANSWER

It assigns a key to the node with the highest deterministic key-node score. Adding a node moves only keys won by the new node, which makes membership changes easier to reason about.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

score = hash(key + delimiter + stableNodeId).
owner = node with the maximum unsigned score.
Use identical encoding and tie-breaking on every client.

WHY THIS WORKS

The simple method checks each node, so cost grows with membership size. Weighted versions can account for unequal capacity. The Java lab verifies the limited-remapping property when adding one node, but it does not implement distributed migration or failure detection.

COMMON PITFALL Java hashCode is not a complete stable cross-language routing specification.

INTERVIEWER FOLLOW-UP

How will routing configurations be versioned during membership changes?

Answer: Distribute a routing epoch; old clients must refresh or be rejected/redirected during rebalance.

How can probabilistic structures and approximate counters be used responsibly?

SHORT INTERVIEW ANSWER

Use approximation where the product can tolerate bounded error, document the error model, and keep correctness-critical decisions on an authoritative path.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Approximate distinct visitors: useful analytics.
Approximate item popularity: useful ranking signal.
Available money or remaining tickets: enforce exact policy.

WHY THIS WORKS

Error bounds, merge behavior and reset semantics differ by algorithm. A metric approximation can be useful without being valid for billing. Include estimation error and pipeline lag in the dashboard meaning. Avoid claiming exact totals because repeated runs happen to match small test data.

COMMON PITFALL An approximate traffic estimate should not silently determine an exact financial settlement.

INTERVIEWER FOLLOW-UP

Which decisions remain correct when the estimate is wrong?

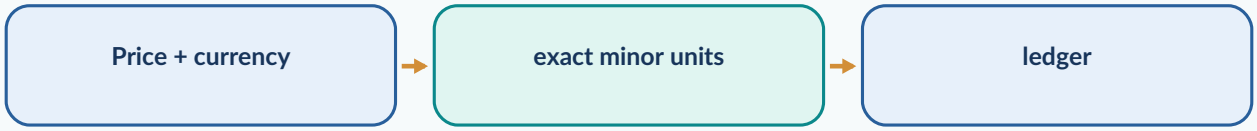
Answer: Use estimates for caching, analytics or hints; never authorize charges, ownership or hard quotas from a guessed count.

How should money be represented in Java and storage?

SHORT INTERVIEW ANSWER

Use a defined currency and exact representation, such as integer minor units or BigDecimal with explicit scale and rounding policy. Do not use binary floating point for authoritative monetary calculations.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
record Money(String currency, long minorUnits) {}
USD 12.34 -> minorUnits 1234 under a two-decimal policy.
Use checked arithmetic and validate matching currencies.
```

WHY THIS WORKS

Currencies and business instruments can have different scales, so do not assume two decimals universally. Overflow, rounding and exchange rates need explicit rules. A ledger records movements and supports reconciliation more reliably than overwriting a balance without history.

COMMON PITFALL BigDecimal constructed from a double can preserve an unintended binary approximation.

INTERVIEWER FOLLOW-UP

At which boundary is tax or exchange rounding applied?

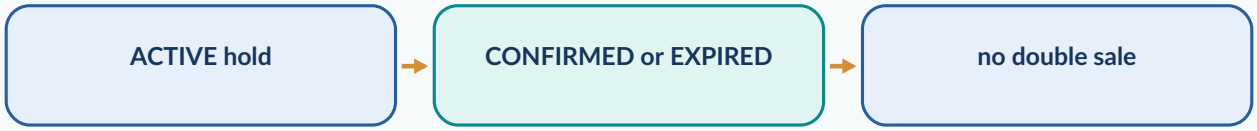
Answer: Round at the defined business stage and currency scale; persist the rounding rule and exact resulting minor units.

Why use reservation states instead of immediately reducing final availability?

SHORT INTERVIEW ANSWER

A reservation models temporary ownership while a workflow completes. Expiry, confirmation and release are explicit transitions with one authoritative decision per resource.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
AVAILABLE -> HELD -> CONFIRMED
HELD -> EXPIRED or RELEASED
Confirm only if hold is active and belongs to the caller.
```

WHY THIS WORKS

An expiry worker and confirmation can race, so the database must arbitrate using conditional updates or locks. The timeout shown to a client should reflect server policy, not only a local countdown. Decide the outcome of payment success after the hold expired.

COMMON PITFALL A scheduled Java timer in one instance is not a durable global expiry mechanism.

INTERVIEWER FOLLOW-UP

Which transition wins if confirmation and expiry execute simultaneously?

Answer: One atomic state transition wins. The losing worker rereads the final status and must not reverse confirmation.

How should payment states handle an unknown outcome?

SHORT INTERVIEW ANSWER

Persist an explicit pending or unknown state, reuse a stable provider key, and query or reconcile before issuing a new effect. A timeout is not evidence that the provider failed.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
CREATED -> SUBMITTED -> SUCCEEDED or DECLINED
SUBMITTED -> UNKNOWN after a lost response
UNKNOWN -> reconciled final state using the same provider reference
```

WHY THIS WORKS

Callbacks can arrive before or after polling results; duplicate and out-of-order updates need a valid transition policy. Keep order state separate from payment state when they have different ownership. Manual review is a valid terminal escalation for unresolved ambiguity.

COMMON PITFALL Automatically charging again after every timeout can create duplicate payments.

INTERVIEWER FOLLOW-UP

How do you handle a success callback after the UI has shown pending?

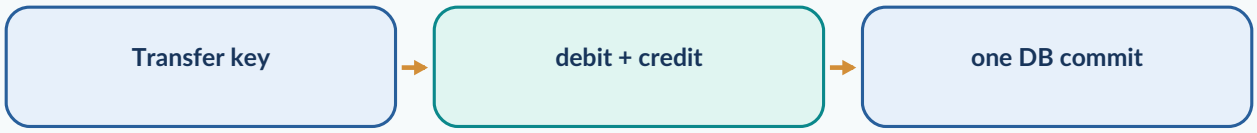
Answer: Keep the order PENDING/RECONCILING, accept a verified late callback idempotently and update the UI safely.

How can a local ledger transfer remain correct under retries?

SHORT INTERVIEW ANSWER

Use a stable command identity, balanced ledger entries and a transaction enforcing account and currency rules. Commit deduplication and the local transfer together.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Command transfer-9:
Debit A by 500; credit B by 500 in the same currency.
UNIQUE(command_id) prevents repeated local application.
```

WHY THIS WORKS

Lock ordering, insufficient funds and serialization conflicts require explicit handling. A transfer to another independent institution is a distributed workflow with settlement and reconciliation, not merely a two-row update. Define whether account balances are derived or materialized and how they are checked.

COMMON PITFALL A globally unique ID alone does not stop duplicate execution unless it is enforced.

INTERVIEWER FOLLOW-UP

How do concurrent debits avoid taking an account below its permitted balance?

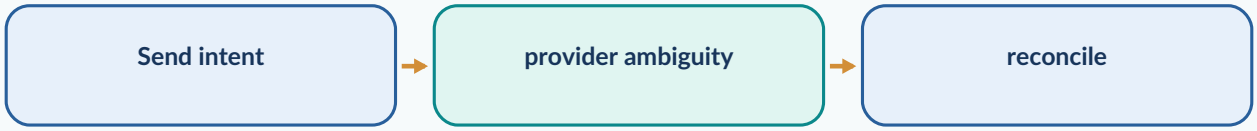
Answer: Lock/conditionally update the balance in one transaction, enforce minimum balance and deduplicate transfer IDs.

What does a notification delivery guarantee actually mean?

SHORT INTERVIEW ANSWER

Accepted, sent to provider, delivered to a device and read by a person are different states. Promise only the state you can observe and control, and retain stable identities across retries.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Notification n-42:
QUEUED -> PROVIDER_ACCEPTED -> DELIVERED when reported.
Read receipts are a separate product behavior.
```

WHY THIS WORKS

Provider callbacks can be duplicated or delayed. Email and SMS may not support perfect deduplication or delivery evidence. Opt-out and preference changes should be checked at the appropriate sending boundary. Define expiration so stale messages do not arrive long after they are useful.

COMMON PITFALL A successful provider HTTP response does not necessarily prove device delivery.

INTERVIEWER FOLLOW-UP

Which state should the dashboard display during provider ambiguity?

Answer: Show 'delivery unknown / checking' rather than 'failed' or 'delivered' until provider evidence arrives.

The cache disappears and the database collapses: what would you change?

SHORT INTERVIEW ANSWER

Treat cache loss as a planned workload, bound fallback concurrency, coalesce reloads and prioritize essential requests. Rebuild gradually instead of flooding the database.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Before: every miss immediately queries DB.
After: bounded loaders + short waits + permitted stale serving.
Cold-cache load test verifies recovery budget.

WHY THIS WORKS

Separate cache misses, cache timeouts and total cache outage in metrics. A circuit breaker on the cache can reduce repeated delays, but the database still needs admission control. Warm-up should avoid all instances loading the same keys simultaneously.

COMMON PITFALL Increasing database connections without checking DB capacity can accelerate collapse.

INTERVIEWER FOLLOW-UP

Which requests will be rejected while the cache warms?

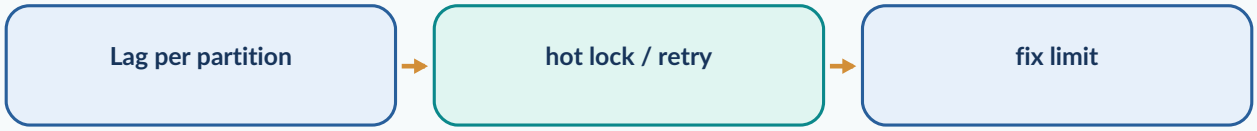
Answer: Reject low-value/nonessential fallback queries first, protect critical checkout and cap DB reload concurrency.

Consumer lag rises while CPU is low: how do you investigate?

SHORT INTERVIEW ANSWER

Measure queue age, partition skew, database or provider waits, retry volume and processing concurrency. Low CPU suggests waiting or uneven work, not necessarily spare usable capacity.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

One partition contains a huge tenant.
Other consumers idle while that partition waits on DB locks.
Adding consumers does not divide this partition automatically.

WHY THIS WORKS

Inspect per-partition lag and duration, not only total backlog. Check ordering constraints before parallelizing effects. A poison event can block progress, while repeated transient failures can consume the entire retry budget. Fix the bottleneck and test backlog recovery alongside live traffic.

COMMON PITFALL Scaling consumers beyond partition or downstream limits may create no throughput gain.

INTERVIEWER FOLLOW-UP

Can the hot tenant be repartitioned without breaking ordering?

Answer: Only with a new partition key/mapping and replay plan. Per-entity order must still be defined and preserved.

A deployment produces duplicate orders: where do you look first?

SHORT INTERVIEW ANSWER

Verify stable request identities, durable uniqueness, transaction boundaries and compatibility between versions. Trace one duplicate from client retries through gateways, services and event consumers.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Version A scopes key by customer + request ID.
Version B accidentally scopes by instance + request ID.
Mixed deployment can admit the same command twice.

WHY THIS WORKS

Compare the actual persisted records and constraint definitions, not just code annotations. Check whether keys expired, payload fingerprints changed or a proxy interception was bypassed. Stop expansion of the bad version and reconcile effects according to business policy.

COMMON PITFALL Deleting duplicates without understanding payment and inventory effects can corrupt state.

INTERVIEWER FOLLOW-UP

Which database constraint would have prevented the regression?

Answer: A unique constraint on the real caller scope and stable request key would reject the duplicate even across versions.

What should a strong design tradeoff explanation sound like?

SHORT INTERVIEW ANSWER

Name the chosen guarantee and its cost, describe the alternative, and connect the choice to a requirement. Avoid presenting technologies as universally better.

DRAW THE IDEA trace the decision, not just the technology



WORKED EXAMPLE

Choice: route stock confirmation to one write authority.
Benefit: one correct reservation ordering.
Cost: higher remote latency and unavailable writes on lost authority.
Alternative: regional stock allocation with explicit budgets.

WHY THIS WORKS

The alternative can improve local availability but adds allocation and rebalancing complexity. Quantify the assumptions that make the decision reasonable. Describe the trigger for revisiting it, such as regional demand or a tighter latency target.

COMMON PITFALL Saying 'it scales better' without naming the bottleneck and cost is incomplete.

INTERVIEWER FOLLOW-UP

What changed requirement would make you choose the alternative?

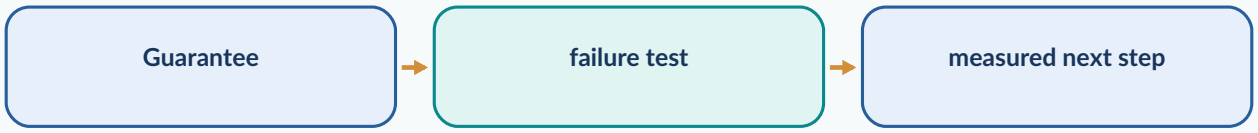
Answer: Choose regional allocation when low local write latency or regional independence becomes a hard requirement and quotas can be split.

How should you finish an interview design answer?

SHORT INTERVIEW ANSWER

Summarize the user journey, guarantees, bottleneck, failure behavior and next evolution. Check that every important requirement maps to an explicit mechanism and observable signal.

DRAW THE IDEA trace the decision, not just the technology



CODE / QUERY EXAMPLE

```
Booking guarantee -> conditional DB transition.
Retry guarantee -> durable request key and stored result.
Recovery -> outbox, reconciliation and tested restoration.
Scale -> indexed reads, cache, partitioning when justified.
```

WHY THIS WORKS

Invite a deep dive into the highest-risk boundary if time remains. Explain one limitation honestly and how you would validate it before production. The goal is a defensible system with clear assumptions, not an elaborate diagram filled with tools.

COMMON PITFALL Do not finish with an unsupported claim that the design has no single point of failure.

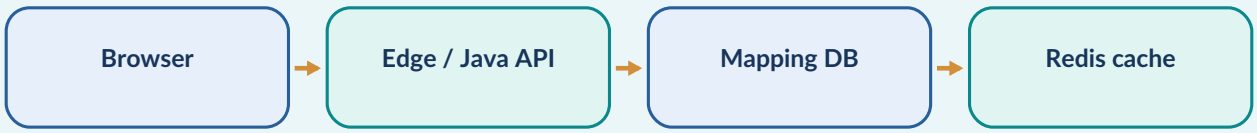
INTERVIEWER FOLLOW-UP

What experiment would you run first before implementing this system?

Answer: Run a concurrent invariant test or failure-injection prototype around the riskiest component, such as last- seat booking.

URL shortener

Design creation and redirect for short URLs, with expiry and click analytics.



REQUIREMENTS AND GUARANTEE

Create a short code, resolve it, support optional expiry and a custom alias. Redirect should be fast; click analytics can be delayed. The code must identify one mapping, and moderation or revocation must have a defined propagation delay.

CAPACITY ASSUMPTIONS (HYPOTHETICAL)

Assume 2 million new links/day and 20 million redirects/day, with an 8x peak. This is approximately 185 peak writes/s and 1,852 peak reads/s. At 500 bytes/mapping, a year of new mappings is approximately 365 GB before indexes, replicas and backups. These are planning assumptions.

FOLLOW THE NORMAL REQUEST

Creation validates the destination and allocates a code, inserts the mapping under a unique constraint, and returns it after commit. Redirect checks permitted edge/cache entries and falls back to the indexed code lookup. It verifies state and expiry before returning the destination. Cache policy must bound revocation lag. A permanent redirect may be cached aggressively by clients, so choose redirect semantics to match whether destinations may change.

Interview technique: point to the one owner that can make the authoritative decision.

URL shortener: make it reliable

API CONTRACT

```
Resource IDs and cursors always require
authorization.
POST /links { destination, expiresAt,
alias? }
GET /{code} -> chosen redirect status +
Location
DELETE /links/{code} -> authorized
revocation
```

DATA AND OWNERSHIP

```
These are logical schema sketches;
constraints and transitions must be
enforced by the authoritative owner.
links(code PK, destination, owner_id, state,
expires_at, version)
UNIQUE(code); lookup by code
outbox(event_id PK, aggregate_id, payload,
published_at)
```

FAILURE PLAYBOOK

DESIGNED RESPONSE Alias collision Unique insert fails; return conflict or retry generated code. Cache unavailable Bound DB fallback and shed excess redirect load. Revoked link cached Use short acceptable TTL, versioned invalidation and purge policy. Analytics backlog Protect redirect path; track lag or dropped telemetry by contract.

TRADEOFF

Sequential IDs encoded with Base62 simplify allocation but expose patterns. Random codes need collision handling and appropriate randomness. Start with one indexed authoritative store and a cache; partition only when measured capacity or data size requires it. Never treat the short code as authorization for a private link.

OBSERVE

Redirect p99 and error rate; cache hit/miss and fallback concurrency; mapping creation conflicts; revocation propagation time; analytics lag.

NEXT EVOLUTION

At 10x read load, expand cache/edge capacity and test cold-cache protection. At larger storage or write scale, hash code placement and plan online migration. Add abuse controls independently of encoding.

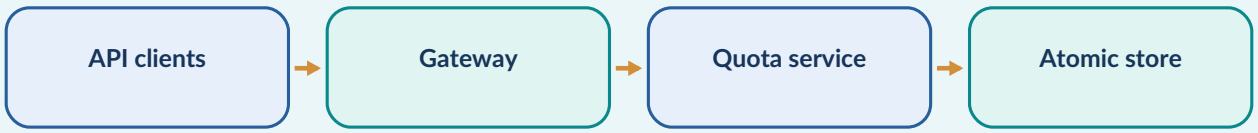
TWO SENIOR FOLLOW-UPS WITH ANSWERS

1. If a destination is edited, how quickly must every cached redirect change?
Answer: Define maximum revocation lag; use short TTLs and explicit invalidation when edits must take effect sooner.

2. How will you stop redirects to malicious destinations?
Answer: Validate destinations at create and redirect, use reputation/moderation checks and emergency revocation.

Distributed rate limiter

Protect a multitenant API with burst allowance and per-tenant quotas.



REQUIREMENTS AND GUARANTEE

Apply tenant and operation policy, allow a controlled burst, return a useful rejection, and prevent one tenant from exhausting shared work. Decide acceptable quota error and outage behavior per endpoint before choosing coordination.

CAPACITY ASSUMPTIONS (HYPOTHETICAL)

Assume 20,000 requests/s across 10,000 active quota keys. If each request performs one remote atomic quota operation, the store must sustain at least that operation rate plus headroom. At an assumed 200 bytes/key, raw state is 2 MB, excluding substantial store overhead. Measure the hot-key distribution and network latency.

FOLLOW THE NORMAL REQUEST

Resolve the validated tenant and policy version. Atomically refill and consume tokens for the key. Allow only if all mandatory policies pass, using a clearly defined multi-policy accounting rule. Reject excess work with 429 where appropriate. Enforce local concurrency separately. If quota storage fails, apply the endpoint-specific degraded policy and monitor it.

Interview technique: point to the one owner that can make the authoritative decision.

Distributed rate limiter: make it reliable

API CONTRACT

```
Resource IDs and cursors always require
authorization.
admit(tenant, operation, cost,
policyVersion)
-> allowed, retryDelay, remainingEstimate
HTTP rejection: 429 with a suitable Retry-
After contract
```

DATA AND OWNERSHIP

```
These are logical schema sketches;
constraints and transitions must be
enforced by the authoritative owner.
quota key: typed tenant + operation + policy
version
state: tokens, refill timestamp, policy
capacity and rate
expiry: remove inactive state only with
defined refill semantics
```

FAILURE PLAYBOOK

DESIGNED RESPONSE State store timeout Apply explicit fail-open/closed policy and local cap. One hot tenant Serialize its quota key; consider allocated node budgets if error is allowed. Clock disagreement Use an agreed server time domain or a defined clock protocol. Policy change Version policy and specify whether remaining tokens carry over.

TRADEOFF

A shared atomic decision gives tighter enforcement but adds latency and a dependency. Local leased budgets reduce calls at the cost of temporary over- or under-admission and recovery complexity. Separate rate, concurrent work, request cost and paid-provider budgets; one number cannot enforce all four.

OBSERVE

Allowed/rejected by bounded policy labels; admission latency; state-store errors; degraded-policy duration; hottest quota keys; downstream saturation.

NEXT EVOLUTION

For larger load, partition by quota key and cache policy. If remote checks dominate latency, quantify acceptable error before allocating local budgets. Load-test simultaneous expiry and a store outage.

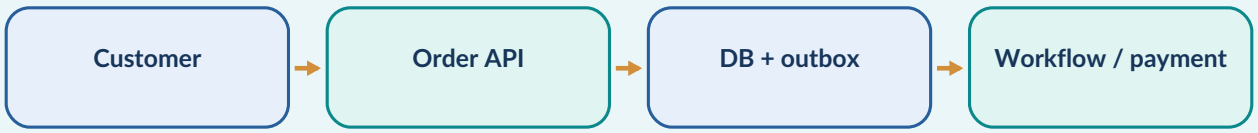
TWO SENIOR FOLLOW-UPS WITH ANSWERS

1. What happens when two quotas must pass but one is consumed before the other rejects?
Answer: Evaluate mandatory quotas atomically together, or implement rollback/reservation; do not silently charge the first quota.

2. Can a regional outage temporarily increase a global tenant quota?
Answer: Yes if regions use independent budgets. Either coordinate a global owner or document bounded temporary overshoot.

E-commerce checkout

Create an order without overselling or charging twice when clients retry.



REQUIREMENTS AND GUARANTEE

Accept an idempotent checkout command, reserve stock, coordinate payment, expose pending/final status and reconcile unknown outcomes. A cart or search result can be stale; final price and inventory decisions must use authoritative policy.

CAPACITY ASSUMPTIONS (HYPOTHETICAL)

Assume 2,000 checkout commands/s at peak, average four items/order and 5 KB of order-plus-items payload. Raw new order data is approximately 10 MB/s before indexes and copies. At an assumed 200 ms mean request duration, about 400 requests are in flight. Payment latency and per-product contention need separate measurements.

FOLLOW THE NORMAL REQUEST

Validate customer and cart, calculate the authoritative quote and store a command-scoped order in PENDING. Commit its outbox record locally. A workflow reserves each inventory item through idempotent commands and authorizes payment with a stable provider key. It confirms the order only after required outcomes are known. On failure it releases reservations or follows the documented compensation policy; on timeout it remains pending and reconciles.

Interview technique: point to the one owner that can make the authoritative decision.

E-commerce checkout: make it reliable

API CONTRACT

```
Resource IDs and cursors always require
authorization.
POST /checkouts + Idempotency-Key
-> orderId, status, statusUrl
GET /orders/{id}; GET /orders/{id}/payment-
status
```

DATA AND OWNERSHIP

```
These are logical schema sketches;
constraints and transitions must be
enforced by the authoritative owner.
orders(id PK, customer_id, request_key,
request_hash, state)
UNIQUE(customer_id, request_key)
reservations(request_id UNIQUE, product_id,
qty, state)
payment_attempt(order_id, provider_key
UNIQUE, state, reference)
```

FAILURE PLAYBOOK

DESIGNED RESPONSE Duplicate command Return same order or conflict for different validated intent. Stock reserved, payment declined Release via idempotent compensation. Payment timeout Persist UNKNOWN/PENDING; query original provider reference. Worker crash Resume durable state; redelivery uses stable step keys.

TRADEOFF

A modular monolith can reserve stock and create an order in one local transaction when ownership is shared. Splitting ownership enables independent scaling but introduces sagas. Do not hold a DB lock across a provider call. Define whether authorization precedes final capture and what refunds mean for the product.

OBSERVE

Checkout success and pending age; stock conflicts; duplicate-key conflicts; provider latency and unknown outcomes; outbox age; compensation backlog.

NEXT EVOLUTION

Partition orders by a suitable ownership key and isolate hot products. Separate analytics from checkout writes. Add inventory allocation by region only with explicit stock budgets and transfer policy.

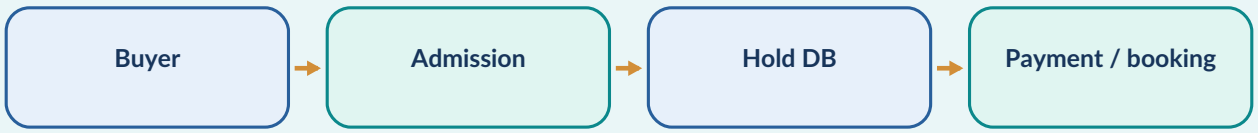
TWO SENIOR FOLLOW-UPS WITH ANSWERS

1. What if payment succeeds after the reservation expired?
Answer: Keep payment outcome pending; reconcile with provider and refund or re-reserve under documented policy.

2. What is the customer-visible state if compensation repeatedly fails?
Answer: Show a pending/manual-review state, retry compensation safely and alert until the business invariant is restored.

Ticket booking and expiring holds

Sell assigned seats fairly under a burst while preventing duplicate ownership.



REQUIREMENTS AND GUARANTEE

Browse availability, hold selected seats for a limited time, confirm on a known payment outcome, and expire unused holds. One confirmed seat has one owner. Define fairness, bot controls and behavior when payment finishes near hold expiry.

CAPACITY ASSUMPTIONS (HYPOTHETICAL)

Assume 50,000 people arrive in one minute: about 833 arrivals/s on average, with a larger stated burst to test. If 5% attempt a hold, that is roughly 42 hold commands/s on average, but a popular section may concentrate most writes. Capacity depends on contention, not merely total QPS. Buyers Admission / queue Search projection Java booking API Seat / hold DB Payment workflow Expiry worker Search may be stale; the seat database decides ownership. Admission protects burst capacity. Expiry and confirmation compete through one authoritative transaction policy.

FOLLOW THE NORMAL REQUEST

Admit a bounded number of buyers and authorize the request. Lock requested seats in a consistent order or use an equivalent atomic protocol. Verify availability and create one hold with an authoritative expiry in the same transaction. Confirm only an active owned hold. An expiry worker releases eligible holds through conditional transitions. Display countdowns as estimates of server expiry and return the authoritative result of confirmation.

Interview technique: point to the one owner that can make the authoritative decision.

Ticket booking and expiring holds: make it reliable

API CONTRACT

```
Resource IDs and cursors always require
authorization.
POST /events/{id}/holds { seatIds } +
request key
POST /holds/{id}/confirm + stable
confirmation key
GET /holds/{id} -> state, expiresAt,
confirmedBookingId
```

DATA AND OWNERSHIP

```
These are logical schema sketches;
constraints and transitions must be
enforced by the authoritative owner.
seat(event_id, seat_id, state, hold_id,
booking_id)
PRIMARY KEY(event_id, seat_id)
hold(id PK, owner_id, state, expires_at,
request_key UNIQUE)
booking(id PK, hold_id UNIQUE,
payment_reference)
```

FAILURE PLAYBOOK

DESIGNED RESPONSE Two buyers choose same seat Only one authoritative hold succeeds. Confirm races expiry Conditional transaction serializes the allowed transition. Expiry worker down Confirmation still checks expiry; sweep catches up later. Payment arrives too late Use declared refund/reallocation/ manual-review policy.

TRADEOFF

A global strict FIFO waiting room can be costly and must define reconnect fairness. Practical admission can be approximate while seat ownership remains exact. Multi-seat holds must be all-or-nothing under the declared contract. A row lock is often simpler than a distributed lease for this local invariant.

OBSERVE

Hold conflict rate; admission wait; booking confirmation latency; expired hold cleanup age; late payment count; confirmed-seat invariant violations.

NEXT EVOLUTION

Partition by event when operations are event-local, reserve capacity for expiry and reconciliation, and load-test one blockbuster event. Cache browsing independently of ownership decisions.

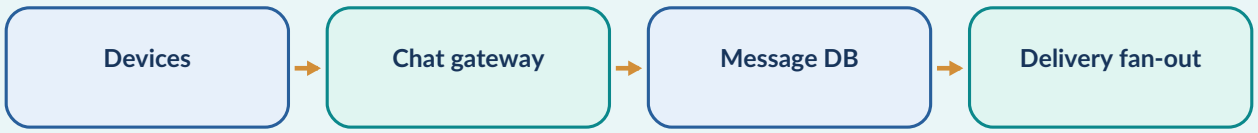
TWO SENIOR FOLLOW-UPS WITH ANSWERS

1. How do you hold four adjacent seats atomically?
Answer: Conditionally claim all four seat records in one short DB transaction; on any conflict roll back every hold.

2. What should a buyer see if the hold expires while the payment page is open?
Answer: Display expiry clearly and verify the authoritative hold state before confirming or charging.

Durable chat

Support multi-device chat, offline history and reconnect without duplicate messages.



REQUIREMENTS AND GUARANTEE

Accept a message once under a stable client key, assign conversation order, persist history, deliver live when possible and resume missed messages. Separate accepted, delivered and read states. Huge public rooms need a different fan-out budget from small conversations.

CAPACITY ASSUMPTIONS (HYPOTHETICAL)

Assume 1 million daily active users sending 20 messages/day: 20 million/day, or about 231/s average and 2,315/s at a stated 10x peak. At 1 KB/message, raw message bytes are about 20 GB/day before replicas and indexes. Separately assume 200,000 concurrent sockets to size gateways. Client devices Socket gateways Connection directory Message owner Message DB / outbox Delivery log Delivery worker The DB is the history authority. The delivery-log arrow includes an outbox relay. The connection directory is ephemeral; failed live routing is repaired by history resume.

FOLLOW THE NORMAL REQUEST

Authenticate the connection and authorize conversation membership. Accept clientMessageId under sender scope, assign a server sequence and commit message plus outbox. Deliver the resulting event to current gateways and devices. Each client deduplicates by server message ID and detects sequence gaps. On reconnect, request history after the last acknowledged sequence. Read receipts follow a separate idempotent state policy.

Interview technique: point to the one owner that can make the authoritative decision.

Durable chat: make it reliable

API CONTRACT

```
Resource IDs and cursors always require
authorization.
send(conversationId, clientMessageId, body)
-> serverId, sequence
GET /conversations/{id}/messages?
afterSequence=104
ackDelivered(serverId, deviceId);
markRead(upToSequence)
```

DATA AND OWNERSHIP

```
These are logical schema sketches;
constraints and transitions must be
enforced by the authoritative owner.
message(conversation_id, sequence,
server_id, sender_id, body)
UNIQUE(conversation_id, sequence)
UNIQUE(sender_id, client_message_id)
connection directory: user -> gateway/
session entries with TTL
```

FAILURE PLAYBOOK

DESIGNED RESPONSE Socket lost after send Retry stable client key and recover accepted server message. Gateway crashes Reconnect and resume from durable conversation history. Delivery out of order Detect gap; fetch and order by server sequence. Slow device Bound buffers; disconnect and resume durable messages.

TRADEOFF

Per-conversation order is often enough; global order across all chats adds needless coordination. A very large room may require relaxed ordering or specialized partitioning. End-to-end encryption changes server search, scanning and key-distribution capabilities and should be treated as a separate requirement.

OBSERVE

Acceptance latency; durable history errors; connection count and buffer bytes; delivery lag; reconnect rate; missing-sequence repairs; hot conversations.

NEXT EVOLUTION

Shard message ownership by conversation, scale gateways independently, and use region-aware routing without creating conflicting conversation writers. Define retention and deletion across history and attachments.

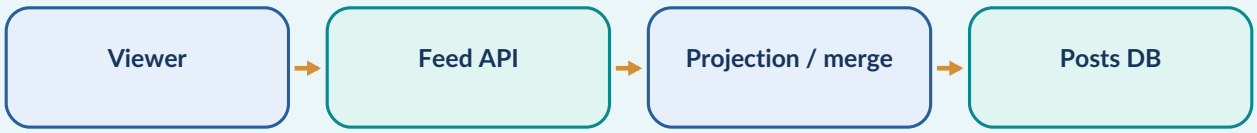
TWO SENIOR FOLLOW-UPS WITH ANSWERS

1. How can a user send from two devices at once?
Answer: Assign per-conversation ordering at its owner and deduplicate independent device request IDs.

2. What does delivered mean when one device receives the message but another is offline?
Answer: Define per-device vs account-level delivery; one online device does not mean every device received it.

Social feed

Build a personalized feed with popular authors and stable pagination.



REQUIREMENTS AND GUARANTEE

Create posts, follow authors, retrieve a useful feed, propagate deletion and enforce privacy. New posts may appear after a defined lag. Ranking, chronological order and pagination behavior must be clarified.

CAPACITY ASSUMPTIONS (HYPOTHETICAL)

Assume 10 million users, 10% daily active and 20 feed reads each: 20 million reads/day. If an ordinary post has 500 followers but a celebrity has 10 million, average fan-out hides the dominant worst case. Model the audience distribution, read frequency and allowable freshness. Reader / author Java feed API Post owner DB Feed cache Fan-out log Projection worker Feed store Follow graph Post-to-log publication uses durable change capture or an outbox. Ordinary author posts populate feed projections; high-audience posts can be merged at read time.

FOLLOW THE NORMAL REQUEST

Commit the authoritative post and change intent. Fan-out workers obtain an appropriate follower view and add stable post IDs to recipient feed projections with duplicate-safe writes. Feed reads fetch the projection, merge designated high-audience sources and apply ranking and authorization. Use a cursor tied to the chosen ranking/snapshot policy. Deletion and privacy changes must hide content even if projection cleanup is delayed.

Interview technique: point to the one owner that can make the authoritative decision.

Social feed: make it reliable

API CONTRACT

```
Resource IDs and cursors always require
authorization.
POST /posts; POST /follows; DELETE /posts/
{id}
GET /feed?cursor=... -> items + nextCursor
Cursor binds ordering/ranking version and
caller context.
```

DATA AND OWNERSHIP

```
These are logical schema sketches;
constraints and transitions must be
enforced by the authoritative owner.
post(id PK, author_id, state, visibility,
created_at)
follow(follower_id, author_id) with suitable
indexes
feed_item(user_id, post_id, ordering_key),
UNIQUE(user_id, post_id)
projection position and version tracked for
rebuilds
```

FAILURE PLAYBOOK

DESIGNED RESPONSE Celebrity posts Use hybrid read-time merge instead of massive synchronous fan-out. Worker redelivery Unique feed item prevents duplicated projection entries. Deleted/private post cached Authoritative visibility/tombstone policy limits exposure. Projection lag Report freshness and rebuild safely from a source position.

TRADEOFF

Fan-out on write improves reads but amplifies work and storage. Fan-out on read handles large audiences more cheaply at write time but increases query work. Hybrid thresholds should follow measured cost. Ranked feeds may need a session snapshot or documented drift across pages.

OBSERVE

Feed p99; projection age; fan-out work per post; celebrity query cost; duplicate suppression; deletion propagation; authorization rejections.

NEXT EVOLUTION

Batch projection writes, isolate large fan-out jobs and partition by recipient. Add ranking pipelines only after defining data freshness and a fallback feed policy.

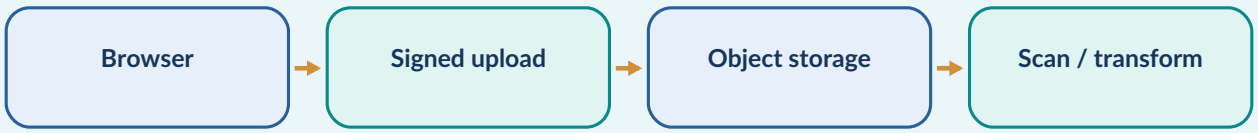
TWO SENIOR FOLLOW-UPS WITH ANSWERS

1. What happens when someone follows an author after a post was published?
Answer: Use a backfill policy or read-time merge for earlier posts; document what new follows can see.

2. How do you avoid repeating items when ranking changes between pages?
Answer: Use a ranking snapshot or stable cursor plus unique post IDs, deduplicating items across pages.

Large file upload and media processing

Accept large user files without filling Java heap, and publish only validated assets.



REQUIREMENTS AND GUARANTEE

Authorize upload, support retry/resume, validate expected content, process safely and serve authorized downloads. An asset is available only after the declared checks complete. Clean up abandoned uploads and orphaned objects.

CAPACITY ASSUMPTIONS (HYPOTHETICAL)

Assume 100,000 uploads/day at 50 MB average: 5 TB/day of raw ingress. Routing every byte through Java APIs adds application bandwidth and memory pressure. Direct object upload separates the control plane from the data plane; transformations, replicas, egress and retention need additional budgets. Uploader Java control API Metadata DB Object storage Finalize event Scan / transform Download / CDN The API issues a scoped upload capability after authorization. Object bytes bypass application heap. Finalization records durable processing intent; publication depends on metadata state and validation.

FOLLOW THE NORMAL REQUEST

Create a draft asset and generate a unique object key. Issue a short-lived upload capability with supported restrictions. The client uploads directly and retries or resumes according to the storage contract. Finalization verifies object ownership, size and checksum and records processing intent. Workers scan and transform into versioned objects, then atomically publish metadata. Downloads check authorization before issuing a capability or serving allowed public content.

Interview technique: point to the one owner that can make the authoritative decision.

Large file upload and media processing: make it reliable

API CONTRACT

```
Resource IDs and cursors always require
authorization.
POST /assets/uploads -> assetId, upload
capability
POST /assets/{id}/finalize -> SCANNING/
PENDING
GET /assets/{id} -> state and authorized
download capability
```

DATA AND OWNERSHIP

```
These are logical schema sketches;
constraints and transitions must be
enforced by the authoritative owner.
asset(id PK, owner_id, object_key UNIQUE,
expected_checksum)
state: UPLOADING, SCANNING, AVAILABLE,
REJECTED, DELETED
transform(asset_id, version, object_key,
state)
processing dedup key: asset + input version
+ transform version
```

FAILURE PLAYBOOK

DESIGNED RESPONSE Client abandons upload Expire draft and clean incomplete/orphaned objects. Upload succeeds, finalize lost Retry stable finalization or reconcile draft object state. Duplicate transform job Use versioned deterministic outputs and an atomic publish decision. Malicious/invalid file Quarantine or reject; never publish before validation policy.

TRADEOFF

Direct upload saves API bandwidth but exposes a capability and introduces two-system reconciliation. Proxying small files can be simpler. Prefer streaming and bounded buffers when proxying is required. Multiple versions make processing and rollback safer but increase storage and cleanup costs.

OBSERVE

Upload/finalize success; draft age; processing lag; rejected scans; checksum failures; orphan bytes; object-store and CDN egress.

NEXT EVOLUTION

Add multipart/resumable upload, processing pools by workload and lifecycle retention. Keep public CDN content separate from private delivery policy. Test interrupted upload and metadata/object mismatch recovery.

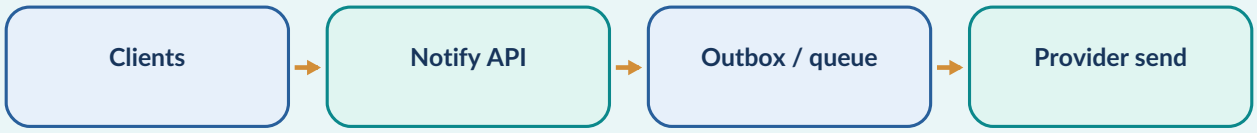
TWO SENIOR FOLLOW-UPS WITH ANSWERS

1. What if a client finalizes a different file than the one authorized?
Answer: Bind the upload grant to object key, owner, size/checksum and validate these again during finalization.

2. How do you prevent a signed URL from overwriting another asset?
Answer: Issue unique, short-lived, scoped object keys; reject overwrites and validate at finalize.

Notification platform

Deliver email, push and SMS under preferences, provider limits and recoverable failures.



REQUIREMENTS AND GUARANTEE

Accept durable intent, choose allowed channels, respect current preferences, rate-limit expensive providers and expose meaningful status. Notifications can expire; provider acceptance is different from device delivery or human reading.

CAPACITY ASSUMPTIONS (HYPOTHETICAL)

Assume 10 million notifications/day: approximately 116/s average and 1,160/s at a stated 10x peak. A provider limited to 500/s will accumulate backlog at 660/s during that peak unless traffic is delayed, rerouted or dropped by policy. Compute backlog duration against notification expiry. Business services Notification API Intent DB / outbox Channel workers Preferences Providers Callbacks / reconcile A local commit accepts notification intent. Channel workers enforce policy and budgets. Callback/reconciliation paths update durable states without assuming every provider response is final delivery.

FOLLOW THE NORMAL REQUEST

Validate and deduplicate a caller-scoped notification intent, then commit it with publish intent. Resolve channel preferences according to the sending policy, render a versioned template and enqueue a channel attempt. Enforce provider rate and concurrency limits, use stable provider keys where supported, and retain ambiguous attempts for reconciliation. Apply duplicate-safe callbacks, expire obsolete work and expose accepted/provider/delivery states separately.

Interview technique: point to the one owner that can make the authoritative decision.

Notification platform: make it reliable

API CONTRACT

```
Resource IDs and cursors always require
authorization.
POST /notifications + caller-scoped request
key
GET /notifications/{id} -> channel attempt
states
Provider callback -> validated provider
event identity
```

DATA AND OWNERSHIP

```
These are logical schema sketches;
constraints and transitions must be
enforced by the authoritative owner.
notification(id PK, caller, request_key,
subject_id, expires_at)
UNIQUE(caller, request_key)
attempt(notification_id, channel,
attempt_key UNIQUE, state)
provider_event(provider, event_id),
UNIQUE(provider, event_id)
```

FAILURE PLAYBOOK

DESIGNED RESPONSE Provider timeout Retain ambiguity; reconcile or retry by provider contract. Preference changed Recheck at sending boundary according to policy. Callback duplicated Persist provider event identity with state transition. Backlog exceeds expiry Expire obsolete intents and prioritize essential classes.

TRADEOFF

Provider failover improves resilience but can create duplicates when the first outcome is unknown. If a provider lacks idempotency or status lookup, document the residual duplicate/loss risk. Keep urgent transactional messages separate from bulk campaigns and ensure quotas preserve fairness.

OBSERVE

Intent acceptance; queue age by bounded channel/class; provider errors; ambiguous attempts; suppression and expiry; reported delivery lag; paid usage.

NEXT EVOLUTION

Partition work by channel and tenant budget, introduce provider-specific adapters and isolate campaigns. Add replay tooling with a dry-run view and explicit duplicate-risk policy.

TWO SENIOR FOLLOW-UPS WITH ANSWERS

1. Can an SMS be sent twice when switching providers after a timeout?
Answer: *Yes: a timeout leaves delivery uncertain, so use provider idempotency where supported and reconcile before fallback.*

2. What should happen to a password-reset message that waited past its useful lifetime?
Answer: *Enforce an expiry or deadline and discard the stale message rather than delivering an unusable reset link.*

Token bucket with an injected clock

Full standalone source: **Lab01.java** · No framework dependencies · `javac --release 17 Lab01.java`

```
public class Lab01 {
    static final class Bucket {
      final double capacity, perSecond;
      double tokens; long lastNanos;
      Bucket(double capacity, double rate, long now) {
        if (capacity <= 0 || rate <= 0 || !Double.isFinite(capacity)
            || !Double.isFinite(rate)) throw new IllegalArgumentException();
        this.capacity = capacity; perSecond = rate;
        tokens = capacity; lastNanos = now;
      }
      synchronized boolean allow(double cost, long now) {
        if (cost <= 0 || !Double.isFinite(cost))
          throw new IllegalArgumentException();
        if (now < lastNanos) throw new IllegalArgumentException("clock");
        tokens = Math.min(capacity,
            tokens + (now - lastNanos) / 1_000_000_000d * perSecond);
        lastNanos = now;
        if (tokens < cost) return false;
        tokens -= cost; return true;
      }
    }
    public static void main(String[] args) {
      var b = new Bucket(2, 1, 0);
      boolean a = b.allow(1, 0), c = b.allow(1, 0);
      boolean blocked = !b.allow(1, 0);
      boolean refill = b.allow(1, 1_000_000_000L);
      boolean oversized = !b.allow(3, 5_000_000_000L);
      boolean invalid = false;
      try { b.allow(0, 5_000_000_000L); }
      catch (IllegalArgumentException ex) { invalid = true; }
      if (!(a && c && blocked && refill && oversized && invalid))
        throw new AssertionError();
      System.out.println("burst=2; blocked=true; refill=true");
      System.out.println("oversized rejected; zero cost rejected");
    }
  }
```

VERIFIED OUTPUT

```
burst=2; blocked=true; refill=true
oversized rejected; zero cost rejected
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

Bounded LRU cache

Full standalone source: **Lab02.java** · No framework dependencies · `javac --release 17 Lab02.java`

```
import java.util.*;
  public class Lab02 {
    static final class Lru<K,V> {
      private final int capacity;
      private final LinkedHashMap<K,V> data =
          new LinkedHashMap<>(16, 0.75f, true);
      Lru(int capacity) {
        if (capacity < 1) throw new IllegalArgumentException();
        this.capacity = capacity;
      }
      synchronized V get(K key) { return data.get(key); }
      synchronized void put(K key, V value) {
        Objects.requireNonNull(key); Objects.requireNonNull(value);
        data.put(key, value);
        if (data.size() > capacity)
          data.remove(data.keySet().iterator().next());
      }
      synchronized List<K> keys() { return List.copyOf(data.keySet()); }
    }
    public static void main(String[] args) {
      var cache = new Lru<String,Integer>(2);
      cache.put("a", 1); cache.put("b", 2); cache.get("a");
      cache.put("c", 3);
      if (cache.get("b") != null) throw new AssertionError();
      if (!cache.keys().equals(List.of("a", "c")))
        throw new AssertionError();
      cache.put("a", 9);
      if (cache.get("a") != 9 || cache.keys().size() != 2)
        throw new AssertionError();
      System.out.println("b evicted; a retained; update does not grow cache");
    }
  }
```

VERIFIED OUTPUT

```
b evicted; a retained; update does not grow cache
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

TTL cache with deterministic expiry

Full standalone source: **Lab03.java** · No framework dependencies · `javac --release 17 Lab03.java`

```
import java.util.*;
  public class Lab03 {
    record Entry(String value, long expiresAt) {}
    static final class Cache {
      private final Map<String,Entry> data = new HashMap<>();
      synchronized void put(String key, String value, long now, long ttl) {
        if (ttl <= 0) throw new IllegalArgumentException();
        data.put(Objects.requireNonNull(key),
            new Entry(Objects.requireNonNull(value), Math.addExact(now, ttl)));
      }
      synchronized String get(String key, long now) {
        var e = data.get(key);
        if (e == null) return null;
        if (now >= e.expiresAt()) { data.remove(key); return null; }
        return e.value();
      }
    }
    public static void main(String[] args) {
      var c = new Cache();
      c.put("p", "v1", 100, 10);
      if (!"v1".equals(c.get("p", 109))) throw new AssertionError();
      if (c.get("p", 110) != null) throw new AssertionError();
      c.put("p", "v2", 110, 20);
      if (!"v2".equals(c.get("p", 120))) throw new AssertionError();
      if (c.get("missing", 120) != null) throw new AssertionError();
      boolean overflow = false;
      try { c.put("x", "x", Long.MAX_VALUE, 1); }
      catch (ArithmeticException ex) { overflow = true; }
      if (!overflow) throw new AssertionError();
      System.out.println("fresh at 109; expired at 110; replacement works");
      System.out.println("missing returns null; expiry overflow rejected");
    }
  }
```

VERIFIED OUTPUT

```
fresh at 109; expired at 110; replacement works
missing returns null; expiry overflow rejected
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

Rendezvous hashing and limited remapping

Full standalone source: **Lab04.java** · No framework dependencies · `javac --release 17 Lab04.java`

```
import java.util.*;
  import java.nio.charset.StandardCharsets;
  import java.security.MessageDigest;
  import java.nio.ByteBuffer;
  public class Lab04 {
    static long score(String key, String node) {
      try {
        byte[] hash = MessageDigest.getInstance("SHA-256")
            .digest((key + "\0" + node).getBytes(StandardCharsets.UTF_8));
        return ByteBuffer.wrap(hash).getLong();
      } catch (Exception ex) { throw new IllegalStateException(ex); }
    }
    static String owner(String key, List<String> nodes) {
      if (nodes.isEmpty()) throw new IllegalArgumentException();
      return nodes.stream().max((a,b) -> {
        int c = Long.compareUnsigned(score(key,a), score(key,b));
        return c != 0 ? c : a.compareTo(b);
      }).orElseThrow();
    }
    public static void main(String[] args) {
      var oldNodes = List.of("A", "B", "C");
      var newNodes = List.of("A", "B", "C", "D");
      int moved = 0;
      for (int i = 0; i < 1000; i++) {
        String key = "k" + i;
        String before = owner(key, oldNodes), after = owner(key, newNodes);
        if (!before.equals(after)) {
          moved++;
          if (!after.equals("D")) throw new AssertionError();
        }
        if (!before.equals(owner(key, List.of("C", "A", "B"))))
          throw new AssertionError();
      }
      if (moved == 0 || moved == 1000) throw new AssertionError();
      System.out.println("all moved keys go to D; list order is irrelevant");
    }
  }
```

VERIFIED OUTPUT

```
all moved keys go to D; list order is irrelevant
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

Executor bulkhead and rejection

Full standalone source: **Lab05.java** · No framework dependencies · `javac --release 17 Lab05.java`

```
import java.util.concurrent.*;
  public class Lab05 {
    public static void main(String[] args) throws Exception {
      var started = new CountDownLatch(1);
      var release = new CountDownLatch(1);
      var pool = new ThreadPoolExecutor(1, 1, 0, TimeUnit.SECONDS,
          new ArrayBlockingQueue<Runnable>(1),
          new ThreadPoolExecutor.AbortPolicy());
      boolean rejected = false;
      Future<Integer> queued = null;
      try {
        pool.execute(() -> {
          started.countDown();
          try { release.await(); }
          catch (InterruptedException ex) { Thread.currentThread().interrupt(); }
        });
        if (!started.await(2, TimeUnit.SECONDS)) throw new AssertionError();
        queued = pool.submit(() -> 7);
        try { pool.submit(() -> 9); }
        catch (RejectedExecutionException ex) { rejected = true; }
        if (!rejected) throw new AssertionError();
      } finally {
        release.countDown(); pool.shutdown();
      }
      if (queued == null || queued.get(2, TimeUnit.SECONDS) != 7)
        throw new AssertionError();
      if (!pool.awaitTermination(2, TimeUnit.SECONDS)) {
        pool.shutdownNow(); throw new AssertionError();
      }
      System.out.println("one running; one queued; third rejected");
      System.out.println("queued work completes after release");
    }
  }
```

VERIFIED OUTPUT

```
one running; one queued; third rejected
queued work completes after release
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

Single-flight loading for concurrent misses

Full standalone source: **Lab06.java** · No framework dependencies · `javac --release 17 Lab06.java`

```
import java.util.concurrent.*;
  import java.util.concurrent.atomic.AtomicInteger;
  import java.util.function.Supplier;
  public class Lab06 {
    static final class Flights {
      final ConcurrentHashMap<String,CompletableFuture<String>> active =
          new ConcurrentHashMap<>();
      final CountDownLatch joined = new CountDownLatch(1); // test observation
      String get(String key, Supplier<String> loader) {
        var mine = new CompletableFuture<String>();
        var existing = active.putIfAbsent(key, mine);
        if (existing != null) { joined.countDown(); return existing.join(); }
        try {
          String result = loader.get(); mine.complete(result); return result;
        } catch (RuntimeException | Error ex) {
          mine.completeExceptionally(ex); throw ex;
        } finally { active.remove(key, mine); }
      }
    }
    public static void main(String[] args) throws Exception {
      var f = new Flights(); var calls = new AtomicInteger();
      var entered = new CountDownLatch(1); var release = new CountDownLatch(1);
      var pool = Executors.newFixedThreadPool(2);
      try {
        var first = pool.submit(() -> f.get("p", () -> {
          calls.incrementAndGet(); entered.countDown();
          try { release.await(); }
          catch (InterruptedException ex) { throw new RuntimeException(ex); }
          return "value";
        }));
        if (!entered.await(2, TimeUnit.SECONDS)) throw new AssertionError();
        var second = pool.submit(() -> f.get("p", () -> {
          throw new AssertionError("duplicate load");
        }));
        if (!f.joined.await(2, TimeUnit.SECONDS)) throw new AssertionError();
        release.countDown();
        if (!second.get(2, TimeUnit.SECONDS).equals("value") ||
            !first.get(2, TimeUnit.SECONDS).equals("value"))
          throw new AssertionError();
        if (calls.get() != 1 || !f.active.isEmpty()) throw new AssertionError();
        System.out.println("two callers; one load; in-flight entry removed");
      } finally { release.countDown(); pool.shutdownNow(); }
    }
  }
```

VERIFIED OUTPUT

```
two callers; one load; in-flight entry removed
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

Local idempotency and conflicting intent

Full standalone source: **Lab07.java** · No framework dependencies · `javac --release 17 Lab07.java`

```
import java.util.*;
  public class Lab07 {
    record Request(String customer, String key, long amount) {}
    record Scope(String customer, String key) {}
    record Stored(Request request, String response) {}
    static final class Service {
      final Map<Scope,Stored> completed = new HashMap<>();
      int effects;
      synchronized String apply(Request r) {
        if (r.amount() <= 0) throw new IllegalArgumentException();
        var scope = new Scope(r.customer(), r.key());
        var old = completed.get(scope);
        if (old != null) {
          if (!old.request().equals(r))
            throw new IllegalArgumentException("key reused with new intent");
          return old.response();
        }
        String result = "order-" + (++effects);
        completed.put(scope, new Stored(r, result));
        return result;
      }
    }
    public static void main(String[] args) {
      var s = new Service(); var r = new Request("c1", "k1", 500);
      String a = s.apply(r), b = s.apply(r);
      boolean conflict = false;
      try { s.apply(new Request("c1", "k1", 700)); }
      catch (IllegalArgumentException ex) { conflict = true; }
      String other = s.apply(new Request("c2", "k1", 500));
      if (!a.equals(b) || a.equals(other) || !conflict || s.effects != 2)
        throw new AssertionError();
      System.out.println("retry returns order-1; conflicting intent rejected");
      System.out.println("other caller has a separate scope; effects=2");
    }
  }
```

VERIFIED OUTPUT

```
retry returns order-1; conflicting intent rejected
other caller has a separate scope; effects=2
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

Concurrent inventory with compare-and-set

Full standalone source: **Lab08.java** · No framework dependencies · `javac --release 17 Lab08.java`

```
import java.util.concurrent.*;
  import java.util.concurrent.atomic.AtomicInteger;
  public class Lab08 {
    static final class Stock {
      final AtomicInteger available = new AtomicInteger(3);
      boolean reserve(int quantity) {
        if (quantity <= 0) throw new IllegalArgumentException();
        while (true) {
          int old = available.get();
          if (old < quantity) return false;
          if (available.compareAndSet(old, old - quantity)) return true;
        }
      }
    }
    public static void main(String[] args) throws Exception {
      var stock = new Stock(); var wins = new AtomicInteger();
      var start = new CountDownLatch(1);
      var pool = Executors.newFixedThreadPool(8);
      try {
        var tasks = new java.util.ArrayList<Future<?>>();
        for (int i = 0; i < 20; i++) tasks.add(pool.submit(() -> {
          try { start.await(); }
          catch (InterruptedException ex) { throw new RuntimeException(ex); }
          if (stock.reserve(1)) wins.incrementAndGet();
        }));
        start.countDown();
        for (var task : tasks) task.get(2, TimeUnit.SECONDS);
        if (wins.get() != 3 || stock.available.get() != 0)
          throw new AssertionError();
        boolean invalid = false;
        try { stock.reserve(-1); }
        catch (IllegalArgumentException ex) { invalid = true; }
        if (!invalid) throw new AssertionError();
        System.out.println("20 callers; 3 reservations; stock=0");
        System.out.println("negative quantity rejected");
      } finally { start.countDown(); pool.shutdownNow(); }
    }
  }
```

VERIFIED OUTPUT

```
20 callers; 3 reservations; stock=0
negative quantity rejected
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

Cursor pagination with equal timestamps

Full standalone source: **Lab09.java** · No framework dependencies · `javac --release 17 Lab09.java`

```
import java.util.*;
  public class Lab09 {
    record Row(long time, long id) {}
    static final Comparator<Row> ORDER =
        Comparator.comparingLong(Row::time).thenComparingLong(Row::id)
            .reversed();
    static List<Row> page(List<Row> rows, Row cursor, int size) {
      if (size < 1) throw new IllegalArgumentException();
      return rows.stream().filter(r -> cursor == null ||
          r.time() < cursor.time() ||
          (r.time() == cursor.time() && r.id() < cursor.id()))
          .sorted(ORDER).limit(size).toList();
    }
    public static void main(String[] args) {
      var rows = List.of(new Row(10,1), new Row(10,2),
          new Row(10,3), new Row(9,4));
      var first = page(rows, null, 2);
      var second = page(rows, first.get(first.size()-1), 2);
      if (!first.equals(List.of(new Row(10,3), new Row(10,2))))
        throw new AssertionError();
      if (!second.equals(List.of(new Row(10,1), new Row(9,4))))
        throw new AssertionError();
      if (!page(rows, second.get(1), 2).isEmpty()) throw new AssertionError();
      System.out.println("page1 IDs=[3, 2]; page2 IDs=[1, 4]");
      System.out.println("same-time rows are neither skipped nor repeated");
    }
  }
```

VERIFIED OUTPUT

```
page1 IDs=[3, 2]; page2 IDs=[1, 4]
same-time rows are neither skipped nor repeated
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

Fencing stale owners at a resource

Full standalone source: **Lab10.java** · No framework dependencies · `javac --release 17 Lab10.java`

```
public class Lab10 {
    static final class Resource {
      long acceptedFence = -1; String value = "initial";
      synchronized boolean write(long fence, String next) {
        if (fence < acceptedFence) return false;
        acceptedFence = fence; value = next; return true;
      }
    }
    public static void main(String[] args) {
      var r = new Resource();
      if (!r.write(7, "A-first")) throw new AssertionError();
      if (!r.write(8, "B-current")) throw new AssertionError();
      if (r.write(7, "A-stale")) throw new AssertionError();
      if (!r.value.equals("B-current")) throw new AssertionError();
      if (!r.write(8, "B-next")) throw new AssertionError();
      System.out.println("fence 8 accepted; stale fence 7 rejected");
      System.out.println("same current owner may write again");
    }
  }
```

VERIFIED OUTPUT

```
fence 8 accepted; stale fence 7 rejected
same current owner may write again
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

Reservation expiry versus confirmation

Full standalone source: **Lab11.java** · No framework dependencies · `javac --release 17 Lab11.java`

```
public class Lab11 {
    enum State { HELD, CONFIRMED, EXPIRED, RELEASED }
    static final class Hold {
      final long expiresAt; State state = State.HELD;
      Hold(long expiresAt) { this.expiresAt = expiresAt; }
      synchronized boolean confirm(long now) {
        if (state == State.CONFIRMED) return true;
        if (state != State.HELD) return false;
        if (now >= expiresAt) { state = State.EXPIRED; return false; }
        state = State.CONFIRMED; return true;
      }
      synchronized boolean expire(long now) {
        if (state == State.HELD && now >= expiresAt) {
          state = State.EXPIRED; return true;
        }
        return false;
      }
    }
    public static void main(String[] args) {
      var timely = new Hold(100);
      if (!timely.confirm(99) || !timely.confirm(101))
        throw new AssertionError();
      if (timely.expire(101)) throw new AssertionError();
      var late = new Hold(100);
      if (late.confirm(100) || late.state != State.EXPIRED)
        throw new AssertionError();
      var swept = new Hold(100);
      if (!swept.expire(100) || swept.confirm(101))
        throw new AssertionError();
      System.out.println("confirmed retry stays confirmed; expiry cannot undo it");
      System.out.println("exact expiry rejects confirmation; sweep wins are final");
    }
  }
```

VERIFIED OUTPUT

```
confirmed retry stays confirmed; expiry cannot undo it
exact expiry rejects confirmation; sweep wins are final
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

Outbox redelivery and local consumer deduplication

Full standalone source: **Lab12.java** · No framework dependencies · `javac --release 17 Lab12.java`

```
import java.util.*;
  public class Lab12 {
    record Event(String id, long amount) {}
    static final class ProducerModel {
      final List<Event> outbox = new ArrayList<>();
      int orders;
      synchronized void create(String eventId, long amount) {
        orders++; outbox.add(new Event(eventId, amount));
      }
    }
    static final class ConsumerModel {
      final Set<String> seen = new HashSet<>(); long total;
      synchronized boolean apply(Event event) {
        if (seen.contains(event.id())) return false;
        long next = Math.addExact(total, event.amount());
        seen.add(event.id()); total = next; return true;
      }
    }
    public static void main(String[] args) {
      var p = new ProducerModel(); var c = new ConsumerModel();
      p.create("e-1", 500);
      var event = p.outbox.get(0);
      if (!c.apply(event) || c.apply(event)) throw new AssertionError();
      if (c.total != 500 || c.seen.size() != 1 || p.orders != 1)
        throw new AssertionError();
      boolean overflow = false;
      try { c.apply(new Event("e-2", Long.MAX_VALUE)); }
      catch (ArithmeticException ex) { overflow = true; }
      if (!overflow || c.seen.contains("e-2") || c.total != 500)
        throw new AssertionError();
      System.out.println("duplicate e-1 ignored; total=500");
      System.out.println("failed effect does not store its dedup marker");
    }
  }
```

VERIFIED OUTPUT

```
duplicate e-1 ignored; total=500
failed effect does not store its dedup marker
```

Practice boundary: This proves the local model's included checks, not a durable multi-node production guarantee. Add concurrency, failure-injection and integration tests before production use.

System guarantees checklist

No overselling	Atomic check-and-reserve in the authoritative store, plus duplicate command handling.
Idempotency	Scoped stable key, validated request intent, durable outcome and in-progress behavior.
Event publication	Business update and outbox intent committed together; relay can redeliver.
Cache consistency	State the stale-data policy; never make stock or money decisions from an untrusted cache.
Fault tolerance	Specify how retries, timeouts, admission and recovery behave during failure.
Regional failure	Name the data owner, failover consistency, RPO and RTO.
Queue processing	Budget backlog age, partitions, redelivery, poison records and consumer effects.
Security	Authorize the resource and operation at its owner, with tenant-scoped queries/cache keys.
Performance	Quantify workload and measure p95/p99, queue wait, pool pressure and bottlenecks.

Strong interview pattern: Requirement → Invariant → Normal request → Failure path → Tradeoff → Validation signal.

30-day preparation plan

Days 01-03	Interview method, capacity, network/API contracts. Write one capacity sheet.
Days 04-06	Scaling, storage, transactions. Demonstrate one atomic stock invariant.
Days 07-09	Sharding, consensus, cache. Explain a split-brain and stale refill race.
Days 10-12	Kafka/queues, outbox, retry safety. Draw a payment timeout state machine.
Days 13-15	Java thread pools, service boundaries, query projections and rebuilds.
Days 16-18	Chat/realtime, auth and tenant isolation, traces/metrics.
Days 19-21	Deployments, DR, multi-region and fairness policies.
Days 22-24	Algorithm choices, booking/money invariants, scenario deep dives.
Days 25-27	Whiteboard URL shortener, booking, chat under 45 minutes each.
Days 28-29	Mock checkout design; full timed mock from random case.
Day 30	Redo difficult labs, recap failure paths and guarantee boundaries.

Study with evidence: All 12 Java labs should be run locally, then modified with extra edge cases. Draw the eight designs from memory and justify at least one rejected alternative.

Lab verification and usage

The companion ZIP contains Lab01.java through Lab12.java, their recorded outputs, a run-all helper, and an interview practice guide.

```
javac --release 17 Lab01.java
java Lab01
# Or run every program with the included script:
bash run_all.sh
```

Lab01	Token bucket with an injected clock
Lab02	Bounded LRU cache
Lab03	TTL cache with deterministic expiry
Lab04	Rendezvous hashing and limited remapping
Lab05	Executor bulkhead and rejection
Lab06	Single-flight loading for concurrent misses
Lab07	Local idempotency and conflicting intent
Lab08	Concurrent inventory with compare-and-set
Lab09	Cursor pagination with equal timestamps
Lab10	Fencing stale owners at a resource
Lab11	Reservation expiry versus confirmation
Lab12	Outbox redelivery and local consumer deduplication

All 12 original lab programs have compiled and executed successfully in this workspace. They test local rules, not live Kafka, Redis, PostgreSQL, regional failover or production contention. Java 21+ virtual-thread behavior is discussed conceptually and is not part of these Java 17 labs.